天津大学

本科生毕业设计（论文）附件

题目：处理器微架构侧信道漏洞安全测试技术研究

学　　院　智能与计算学部

专　　业　软件工程

年　　级　2022 级

姓　　名　杨宇鑫

学　　号　3022207128

指导教师　鲁赵骏

目　录

天津大学

本科生毕业设计（论文）任务书

题目：处理器微架构侧信道漏洞安全测试技术研究

学　　院　智能与计算学部

专　　业　软件工程

年　　级　2022 级

姓　　名　杨宇鑫

学　　号　3022207128

指导教师　鲁赵骏

一、原始依据

近年来，处理器微架构侧信道漏洞引发了学术界与工业界的持续关注。此类问题通常源于高速缓存、分支预测、乱序执行、预取与共享资源仲裁等微架构机制在提升性能的同时引入了可被观测的时间差、功耗差或资源竞争差异，使攻击者能够在不直接突破权限边界的情况下推断敏感信息。随着云计算与多租户虚拟化环境的普及，来自不同安全域的代码可能共享同一物理处理器与缓存层次结构，侧信道风险因此被放大。同时，软件供应链复杂化与第三方代码广泛引入，也使得对平台侧信道暴露面的系统化评估与持续测试成为现实需求。相较于单次漏洞利用研究，面向安全测试的技术更强调可重复、可度量与可自动化的评估流程，能够在系统上线前或更新后及时发现风险并验证缓解措施效果。

本课题具有良好的工作基础与研究条件。学生可基于公开论文与已有的开源测试工具了解典型侧信道模式与测试方法，并在通用 x86 平台上开展实验。实验条件可使用普通实验室计算机或服务器，在 Linux 环境下通过计时指令、性能计数器与微基准程序对缓存与分支行为进行观测，必要时在隔离环境中使用虚拟机进行对照。应用环境面向需要评估处理器与系统软件安全性的场景，例如云主机平台、校内服务器集群、关键业务主机与安全加固验证等。工作目的在于形成一套面向微架构侧信道的安全测试思路与基本工具链，能够对关键场景进行风险检测与效果评估，输出规范的测试流程、实验数据与结论建议，为平台选型、配置加固与安全审计提供依据。

二、参考文献

[1] Graf J C, Ruegge S, Hajiabadi A, et al. VMSCAPE: Exposing and Exploiting Incomplete Branch Predictor Isolation in Cloud Environments[J]. Cited on, 3.

[2] Chiang L C, Li S W. Reload+ Reload: Exploiting Cache and Memory Contention Side Channel on AMD SEV[C]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2025: 1014-1027.

[3] Schlüter B, Wech C, Shinde S. Heracles: Chosen plaintext attack on amd sev-snp[C]//Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 3810-3824.

[4] Muñoz A, Ríos R, Román R, et al. A survey on the (in) security of trusted execution environments[J]. Computers & Security, 2023, 129: 103180.

[5] Pashrashid A, Hajiabadi A, Carlson T E. Efficient Detection and Mitigation Schemes for Speculative Side Channels[C]//2024 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2024: 1-5.

三、设计（研究）内容和要求

本课题以微架构侧信道安全测试为目标，完成可运行的测试用例与评估报告。研究内容包括以下几个方面。第一，选择两类典型侧信道方向开展测试设计，可从缓存计时侧信道与分支相关计时侧信道中各选一类，阅读公开资料后整理其基本原理、测试前提与可观测信号，形成简要技术综述。第二，搭建测试环境，在 Linux 系统下完成计时与采样工具准备，编写或改造微基准程序实现可重复的时间测量与数据采集，并提供一键运行脚本，保证不同轮次的测试流程一致。第三，完成对照测试与缓解验证，在默认配置下采集基线数据，再开启或关闭若干常见缓解措施进行对比，例如进程亲和绑定、缓存刷新指令的使用方式变化，或系统提供的相关隔离选项，观察统计结果变化并给出解释。第四，形成测试结果展示与结论建议，输出图表与统计摘要，给出在所测平台上的风险描述与推荐配置。

主要指标与技术参数建议如下。至少给出时间测量的统计指标，包括平均值、标准差与分位数，并报告样本数量与重复次数。对缓存侧信道测试，可给出命中与未命中两类访问的时间分布差异或可区分度指标。对分支相关测试，可给出不同分支历史下的时间差异统计。对缓解验证需给出对比前后差异幅度。实验参数需明确 CPU 型号与核心数、操作系统版本、编译器版本与优化选项，计时方式可采用高精度时间戳计数器或系统高精度计时接口，并说明测量开销控制方法。

对学生具体要求包括能够使用 C 语言或 Python 进行基础编程与脚本编写，能够在命令行环境完成编译、运行与数据处理，能够严格按实验规范记录环境配置与测试参数。提交内容包括测试代码与脚本、原始数据与绘图结果、测试流程文档与总结报告。研究不要求复现复杂的漏洞利用链条，不要求内核修改或硬件级调试，重点在于将测试流程做成可复现、可对比的实验体系。

指导教师（签字） 

2026 年 1 月 20 日

审题小组组长（签字） 

2026年　1月　20日

天津大学

本科生毕业设计（论文）开题报告

题目：处理器微架构侧信道漏洞安全测试技术研究

学　　院　智能与计算学部

专　　业　　软件工程

年　　级　　2022 级

姓　　名　　杨宇鑫

学　　号　　3022207128

指导教师　　鲁赵骏

一、课题的来源及意义

本课题来源于计算机系统安全领域的实际需求。随着云计算和多核处理器的广泛应用，处理器微架构侧信道攻击已成为信息安全领域的重要威胁。本课题聚焦 Intel 主流处理器平台，面向已有公开漏洞的复现与验证，系统研究不同架构下的侧信道安全特性。研究内容基于 2025 年 S&P 发表的 VMSCAPE 论文，结合经典侧信道案例，开展跨平台的安全测试与对比分析。

二、国内外发展状况

1. 国外发展状况：

自 2005 年首次提出缓存侧信道攻击以来，国外学者在该领域开展了大量研究。Flush+Relad（2013 年）、Spectre（2018 年）等攻击技术的提出，推动了侧信道攻击技术的快速发展。

Intel 平台因其市场占有率高，成为侧信道攻击研究的主要目标。已有大量针对 Intel 处理器的侧信道漏洞被发现和分析，如 Meltdwn、L1TF 等。

部分研究开始关注不同处理器架构的侧信道特性对比，分析架构设计差异对攻击效果的影响。

Intel 处理器厂商通过微码更新、固件升级等方式不断加强处理器安全性。学术界也提出了多种软件层面的缓解措施，如缓存隔离、编译优化等。

2. 国内发展状况：

国内在侧信道攻击领域的研究相对滞后，但近年来发展迅速。部分高校和科研机构已开始关注处理器微架构侧信道安全问题。

国内研究主要集中在 Intel 平台的漏洞分析和复现。

三、研究目标、研究内容与研究方法

1. 研究目标：

跨平台数据采集与分析：使用高精度计时和性能计数器，收集平台的侧信道攻击数据，分析攻击特征和影响因素的差异。

量化评估与对比：设计统一的评测指标，量化平台的攻击泄露带宽、命中率、

信噪比以及跨核/跨进程可迁移性，并进行对比分析。

缓解措施验证与对比：测试常见缓解措施（如隔离策略、缓存冲洗、编译器优化、微码更新等）在平台上的效果，对比实施前后的差异。

2. 研究内容：

侧信道威胁模型分析：

梳理 Intel 处理器的微架构特点，对比分支预测器、缓存系统等与侧信道相关组件的设计差异。

分析 Flush+Relad 和 Spectre-BTI 攻击在平台上的威胁模型和攻击前提。

跨平台实验环境搭建：

搭建基于 Intel 处理器的 Linux 测试环境。

配置高精度计时工具和性能计数器，确保跨平台测试的一致性。

基于 VMSCAPE 论文和开源代码，实现跨平台的攻击原型。

Flush+Relad 攻击复现与跨平台对比：

在平台上实现 Flush+Relad 攻击的核心逻辑，包括缓存冲洗、内存访问时间测量等。

测试不同参数（如槽位数、测量阈值等）对攻击效果的影响，对比平台的差异。

分析攻击的时序特征和成功率，识别平台特定的影响因素。

Spectre-BTI 攻击复现与跨平台对比：

基于 VMSCAPE 论文，在平台上实现 Spectre-BTI 攻击的原型。

测试攻击在不同型号 Intel 处理器上的效果，分析架构差异对攻击的影响。

对比平台上分支目标注入的机制和影响因素。

跨平台评测指标设计与实现：

设计统一的泄露带宽、命中率、信噪比等量化指标，确保跨平台对比的公平性。

开发跨平台数据采集和分析脚本，生成可视化图表，直观展示平台的攻击效果差异。

3. 研究方法：

文献研究法：系统阅读 VMSCAPE 论文及相关文献，了解侧信道攻击的理论基础和最新进展，特别是跨平台对比研究。

实验法：在 Intel 处理器平台上搭建测试环境，复现侧信道攻击，收集实验数据，进行对比分析。

量化评估法：使用设计的评测指标，对平台的攻击效果和缓解措施的有效性进行量化评估和对比。

四、进度安排

2026.01.30 前：完成开题报告撰写与提交。开题报告通过指导教师审核，上传至毕设管理系统。

2026.02.01-2026.02.20：搭建 Intel 平台实验环境。成功搭建 Linux 测试环境，配置好必要的工具和库。

2026.02.21-2026.03.10：复现 Flush+Relad 攻击（两平台）。在平台上实现攻击原型，成功收集侧信道数据。

2026.03.11-2026.03.31：复现 Spectre-BTI 攻击（两平台）。基于 VMSCAPE 论文实现攻击原型，验证攻击在平台上的效果。

2026.04.01-2026.04.10：完成中期检查。提交中期检查报告，展示已完成的跨平台实验工作和初步成果。

2026.04.11-2026.04.30：设计并实现跨平台评测指标。开发数据采集和分析脚本，生成量化结果和可视化对比图表。

2026.05.01-2026.05.15：验证缓解措施效果。测试并分析各种缓解措施在平台上的有效性差异。

2026.05.16-2026.05.25：完成论文撰写与提交。论文通过指导教师审核，上传至毕设管理系统。

2026.05.26-2026.06.06：准备答辩材料。制作答辩 PPT，准备跨平台演示环境。

2026.06.07 前：完成答辩并提交成绩。答辩通过，提交最终成绩。

五、研究或实验方案的可行性分析

1. 技术可行性：

硬件平台：Intel 处理器在市场上广泛可用，实验所需的硬件环境容易搭建。可选择多种型号的处理器进行测试，确保结果的通用性。

软件工具：Linux 操作系统提供了丰富的系统接口和工具，支持高精度计时和性能计数器的使用。VMSCAPE 论文及相关文献提供了详细的攻击实现方法，开源代码也可作为参考。

2. 经济可行性：

硬件成本：实验仅需要两台分别搭载 Intel 处理器的计算机，无需特殊硬件设备，硬件成本较低。

软件成本：使用的操作系统和工具均为开源软件，无需支付额外费用。

3. 风险分析与应对措施：

硬件兼容性风险：不同型号的 Intel 处理器可能存在差异，影响攻击的效果。

应对措施：选择多种型号的处理器进行测试，确保结果的通用性和代表性。

软件环境配置风险：Linux 系统版本和配置可能影响实验结果，跨平台配置差异可能导致测试不一致。

应对措施：使用标准化的测试环境，记录详细的系统配置信息，确保跨平台测试的一致性。

六、参考文献

[1] Graf J C, Ruegge S, Hajiabadi A, et al. VMSCAPE: Exposing and Exploiting Incomplete Branch Predictor Isolation in Cloud Environments[J]. Cited on, 3.

[2] Chiang L C, Li S W. Reload+ Reload: Exploiting Cache and Memory Contention Side Channel on AMD SEV[C]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2. 2025: 1014-1027.

[3] Schlüter B, Wech C, Shinde S. Heracles: Chosen plaintext attack on amd sev-snp[C]//Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. 2025: 3810-3824.

[4] Muñoz A, Ríos R, Román R, et al. A survey on the (in) security of trusted execution environments[J]. Computers & Security, 2023, 129: 103180.

[5] Pashrashid A, Hajiabadi A, Carlson T E. Efficient Detection and Mitigation Schemes for Speculative Side Channels[C]//2024 IEEE International Symposium on Circuits and Systems (ISCAS). IEEE, 2024: 1-5.

[6] Intel. Intel 64 and IA-32 Architectures Sftware Develper's Manual. 2022.

[7] Intel. Mitigatins fr Speculative Executin Side Channels. 2018.

[8] Gruss D, et al. Prefetch Side-Channel Attacks: Bypassing SMAP and Kernel ASLR. USENIX Security Sympsium, 2016.

七、预期成果

1. **学术成果：**

完成一篇详细的毕业设计论文，系统阐述 Intel 平台侧信道安全问题的对比研究成果。

2. **技术成果：**

实现 Flush+Relad 和 Spectre-BTI 攻击的跨平台可复现原型。

开发跨平台数据采集和分析脚本。

设计并实现统一的量化评估指标，用于跨平台对比。

生成可视化图表，直观展示平台的攻击效果和缓解措施差异。

选题是否合适： 是☑ 否☐ 课题能否实现： 能☑ 不能☐ 指导教师（签字）鲁赵毅 2026年 1月 25日
选题是否合适： 是☑ 否☐ 课题能否实现： 能☑ 不能☐ 审题小组组长（签字）鲁赵毅 2026年 1月 25日

天津大学

本科生毕业设计（论文）外文翻译

外文原文题目：

VMSCAPE: Exposing and Exploiting Incomplete Branch Predictor Isolation in Cloud Environments

中文译文题目：

VMSCAPE：在云环境中暴露并利用不完整的分支预测器隔离机制

毕业论文题目：

处理器微架构侧信道漏洞安全测试技术研究

学　　院　智能与计算学部

专　　业　软件工程

年　　级　2022 级

姓　　名　杨宇鑫

学　　号　3022207128

指导教师　鲁赵骏

VMSCAPE: Exposing and Exploiting Incomplete Branch Predictor Isolation in Cloud Environments

Jean-Claude Graf †
ETH Zurich

Sandro Rüegge †
ETH Zurich

Ali Hajiabadi
ETH Zurich

Kaveh Razavi
ETH Zurich

†Equal contribution joint first authors

***Abstract*—Virtualization is a cornerstone of modern cloud infrastructures, providing the required isolation to customers. This isolation, however, is threatened by speculative execution attacks which the CPU vendors attempt to mitigate by extending the isolation to the branch predictor state. Our systematic analysis shows that this extension unfortunately is incomplete: while the most obvious case of the guest controlling branch prediction in the host has been addressed by existing hardware mitigations, we discover a number of new Spectre *Branch Target Injection* (Spectre-BTI) attack primitives on AMD Zen 1-5 and Intel Coffee Lake CPUs that, among others, enable a malicious guest to control indirect branch prediction in the host when it is executing in userspace. Using the aforementioned primitive, we craft VMSCAPE, the first Spectre-BTI attack that enables a malicious KVM guest to leak arbitrary memory from an unmodified QEMU process running on an AMD Zen 5 server system at the speed of 154 B/s, exposing cryptographic keys for disk encryption and decryption. Our analysis of possible mitigation strategies shows that it is possible to mitigate VMSCAPE by selectively flushing the branch predictor with minimal performance impact in common scenarios.**

1. Introduction

Spectre v2 or *Branch Target Injection* (BTI) [1] attacks have received tremendous attention recently, typically leaking privileged memory from an unprivileged user [2], [3], [4], [5], [6], [7], [8]. While these attacks show-case the violation of security boundaries, they have limited real-world impact since they assume local code execution on a user's system. The more interesting threat model is in the cloud where a malicious *Virtual Machine* (VM) can exploit Spectre-BTI to leak information from the host or another VM. The only published attacks in these scenarios, however, need attacker-controlled code in the host [1], [5], [8] which is an unrealistic assumption. Through a systematic analysis of branch prediction state isolation across virtualization boundaries, we discover a number of novel attack primitives based on Spectre-BTI. We then leverage one of these primitives to build the first Spectre-BTI exploit that leaks information from a host that is running unmodified software in default configuration.

Spectre-BTI in the cloud. Virtualization, in the form of VMs, is the primary mechanism for securely isolating co-located workloads in the cloud, protecting both customer data and the underlying infrastructure [9]. Spectre attacks, and in particular Spectre-BTI, can compromise this isolation by abusing the shared branch predictor state inside the CPU [1]. The commonly considered threat models in this scenario are a malicious VM attacking the hypervisor or another VM. As such, the hardware and software vendors give particular attention to mitigating such threats by tagging branch prediction entries learned in the VM and the hypervisor differently and flushing the branch predictor state when switching between different VMs. The residual attack surface through confused-deputy attacks on the hypervisor is difficult to exploit in practice without assuming code changes [5]. The question we investigate in this paper is if there are other unexplored attack primitives that are possible from a malicious VM.

New Spectre-BTI primitives for the cloud. We make a key observation that the existing branch predictor isolation mechanisms are too coarse-grained—they consider the hypervisor and VMs as monolithic entities while in reality they contain their own privilege levels. To investigate how existing branch predictor isolation mechanisms handle these privilege levels in different virtualization domains, we systematically analyze all possible combinations of attacker and victim protection domains in x86 CPUs, namely *Host Supervisor* (HS), *Host User* (HU), *Guest Supervisor* (GS), and *Guest User* (GU). Our analysis demonstrates that, despite existing hardware mitigations, all AMD Zen processors (Zen 1 to Zen 5) and Intel Coffee Lake are vulnerable to new *Virtualization-based Spectre-BTI* (vBTI) attack primitives between guest users and host users which we refer to as $\text{vBTI}_{\text{GU}\rightarrow\text{HU}}$ and $\text{vBTI}_{\text{HU}\rightarrow\text{GU}}$. $\text{vBTI}_{\text{GU}\rightarrow\text{HU}}$ enables new Spectre-BTI attacks on user processes running on the host from a malicious VM, and $\text{vBTI}_{\text{HU}\rightarrow\text{GU}}$ enables new Spectre-BTI attacks in scenarios where the host is assumed to be malicious and the guest has deployed hardware-assisted isolation, such as *Secure Encrypted Virtualization - Secure Nested Paging* (SEV-SNP) [10] or *Trust Domain Extensions* (TDX) [11]. We additionally discover other vBTI primitives that require consideration for other attack scenarios that we will discuss in this paper.

VMSCAPE. To demonstrate the practicality and severity of our vBTI primitives, we present VMSCAPE, the first Spectre-based end-to-end exploit in which a malicious guest user can leak arbitrary, sensitive information from the hypervisor in the host domain, without requiring any code modifications and in default configuration. We target the widely used *Kernel Virtual Machine* (KVM)/QEMU [12], [13] as the hypervisor, and particularly QEMU as the user-space component of the hypervisor in the host. While the $\text{vBTI}_{\text{GU}\rightarrow\text{HU}}$ primitive demonstrates the feasibility of the exploit, we address several additional challenges to enable reliable end-to-end attacks on AMD Zen 4 and Zen 5 CPUs. For instance, achieving arbitrary memory leakage requires a sufficiently large speculation window. However, obtaining a large speculation window is challenging for a guest user, since it cannot simply flush the victim branch pointer due to lack of access to physical memory or code modifications in QEMU. To overcome this, we reverse engineer AMD Zen 4 and Zen 5's cache hierarchy to build eviction sets for their non-inclusive *Last Level Cache* (LLC), thereby enabling a sufficiently large speculation window for VMSCAPE to succeed with the gadgets that we have found in QEMU.

VMSCAPE can leak the memory of the QEMU process at a rate of $154\,\text{B/s}$ on AMD Zen 5. We use VMSCAPE to find the location of secret data and leak it, all within $102\,\text{s}$, extracting the cryptographic key used for disk encryption/decryption as an example. Our analysis shows that mitigating VMSCAPE requires an explicit flush of the *Branch Prediction Unit* (BPU) state in certain conditions. In particular, an *Indirect Branch Prediction Barrier* (IBPB) is necessary on each `VMEXIT` before entering the userspace hypervisor. Our evaluation shows that such a mitigation, as developed by the Linux kernel maintainers as a response to VMSCAPE, introduces a marginal performance overhead in common scenarios.

Contributions. We make the following contributions:

- The first systematic analysis of branch predictor isolation across all possible virtualization and privilege boundaries, leading to the discovery of new primitives that are effective even with recent CPU mitigations.
- The first guest-to-host Spectre-BTI attack on unmodified software, called VMSCAPE, using our newly-discovered $\text{vBTI}_{\text{GU}\rightarrow\text{HU}}$ primitive and new insights on cache evictions on AMD Zen 4 and Zen 5 for increasing the speculation window.
- An analysis of mitigation possibilities on various microarchitectures, leading to IBPB-on-`VMEXIT` as a basis for mitigating VMSCAPE.

Responsible disclosure. We have disclosed VMSCAPE to AMD and Intel PSIRT on June 7, 2025, and the issue remained under embargo till September 11, 2025. Linux maintainers mitigated VMSCAPE with patches based on our IBPB-on-`VMEXIT` recommendation. We verified that these patches are effective in stopping our vBTI primitives. VMSCAPE is tracked under CVE-2025-40300. Further information, including the source code of our experiments and exploit can be found at https://comsec.ethz.ch/vmscape.

2. Background

We provide some background on the *Kernel Virtual Machine* (KVM)-based virtualization stack in Linux (Section 2.1), branch prediction mechanisms (Section 2.2), Spectre-BTI attacks (Section 2.3), and their respective mitigations (Section 2.4). We conclude by motivating the need to explore Spectre-BTI attacks in the cloud scenario (Section 2.5).

2.1. KVM-Based Virtualization

Hardware virtualization extensions such as AMD SVM [10] and Intel VT-x [14] provide hardware support for running multiple operating systems concurrently on the same processor, offering strong isolation guarantees with minimal performance overhead. The *hypervisor* presents each *guest Virtual Machine* (VM) with the illusion of full control over a physical system. On Linux, KVM acts as the hypervisor and is implemented as a kernel module running on the *host*. KVM relies on a user-space component responsible for managing the lifecycle and emulated devices of a VM. We refer to this user-space process as *User-Mode Component of the Hypervisor* ($\text{Hypervisor}_{\text{user}}$). QEMU [13] is a widely-used $\text{Hypervisor}_{\text{user}}$ in Linux systems [15]. It performs tasks such as creating, starting, stopping, and migrating VMs, and emulates required hardware devices. Each guest *virtual CPU* (vCPU) maps to a dedicated thread in the $\text{Hypervisor}_{\text{user}}$, allowing the host kernel to schedule guest execution just like normal host processes since threads and processes are treated similarly by the scheduler in Linux. Consequently, the executions of the guest threads are interleaved with other guest threads and host processes.

Entering and exiting VMs. A vCPU thread either runs in "guest-mode" or "host-mode". To transition from host-mode to guest-mode, it issues a `KVM_RUN` *ioctl* to the KVM kernel module, passing a pointer to its vCPU. Depending on the platform, KVM either issues the `VMLAUNCH/VMRESUME` (on Intel) or `VMRUN` (on AMD) instruction. The CPU restores its state from the vCPU and starts the guest execution. When the guest executes a sensitive or privileged instruction, the CPU intercepts it, resulting in a `VMEXIT`. The CPU saves its internal state back into the vCPU structure and continues execution in KVM. Depending on the `VMEXIT` type, KVM either handles the exit directly or delegates it to the $\text{Hypervisor}_{\text{user}}$ through a `KVM_EXIT`.

Protection domains. To ensure confidentiality and integrity, computing systems must enforce isolation between entities operating in different protection domains. In many contexts, the focus is on the distinction between the *user* and *supervisor* domains (i.e. ring 0 vs ring 3). However, when considering virtualized environments, this abstraction must be extended to account for guest execution. As illustrated in Figure 1, the user domain then comprises both *Host User* (HU) and *Guest User* (GU) domains, while the supervisor domain encompasses *Host Supervisor* (HS) and *Guest Supervisor* (GS) domains.

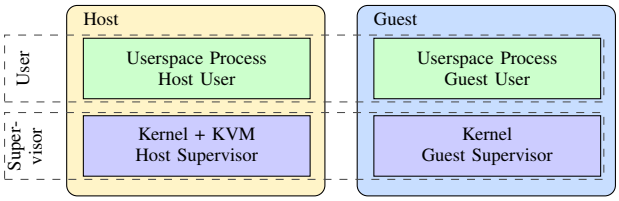


Figure 1. On x86-64 systems with VMs, protection domains include HU, HS, GU, and GS, extending beyond user and supervisor modes.

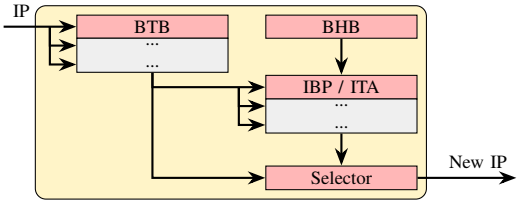


Figure 2. BPU components involved in predicting indirect branches. A BTB provides static predictions, while the IBP/ITA correlate targets with path history stored in the BHB on Intel/AMD.

2.2. Branch Prediction

A cornerstone optimization of CPUs is the accurate prediction of branch instructions and their targets. The *Branch Prediction Unit* (BPU) in modern CPUs is highly complex, composed of different predictors tailored to different branch types and contexts. While *static predictors* only consider the source address of a branch, *dynamic predictors* use the path history of a branch as additional context [2], [3], [6], [7], [16], [17]. *Indirect branches* impose a control flow transfer to a destination either given by a register or through memory. This allows them to have a multitude of destinations that the BPU tries to correlate with the path history of the branch. Intel CPUs use a *Branch Target Buffer* (BTB) for static predictions and an *Indirect Branch Predictor* (IBP) for dynamic predictions. When available, dynamic predictions take priority over static predictions [8], [17]. On AMD CPUs, the dynamic predictor is called *Indirect Target Array* (ITA), and it is used when a branch has encountered multiple targets. Otherwise only the BTB is used [18]. Path history is recorded in the *Branch History Buffer* (BHB) on both Intel and AMD. Figure 2 illustrates the structures involved when predicting indirect branches.

2.3. Spectre-BTI

Spectre Branch Target Injection or Spectre-BTI [19] is a class of speculative execution attacks that allow an attacker to leak data across protection domain boundaries by exploiting indirect branch mispredictions. The attacker causes the BPU to predict the *victim branch* (as part of a *speculation gadget*) to a *disclosure gadget*, inducing *transient execution*. In the disclosure gadget, a secret is accessed in a way that lets the attacker later infer it through a *side channel*, typically a cache-based FLUSH+RELOAD [20]. To cause a misprediction of the victim branch, the attacker *trains* the BPU using a *training branch* that *collides* with the victim branch in the targeted predictor. To target a specific predictor, the attacker may also need to mimic the path history of the victim branch in the training branch.

Most prior work on Spectre-BTI has focused on scenarios in which an unprivileged host user process targets the host supervisor [1], [2], [3], [5], [6], [21]. In contrast, attacks between user processes have received relatively little attention [7].

2.4. Spectre-BTI Mitigations

Spectre-*Branch Target Injection* (BTI) attacks require the attacker to have control over the BTB state that is used to predict the victim, implying the need for shared BPU state between the attacker and victim domains. *Domain isolation* techniques aim to mitigate BTI attacks by preventing the sharing of BPU state across critical protection domain boundaries. *Indirect Branch Restricted Speculation* (IBRS) [22] prevents branches of lower privileged domains (e.g., user) from influencing branches of higher privileged domains (e.g., supervisor), thereby defending the kernel from user-mode attackers. Intel and AMD have deployed *Enhanced IBRS* (eIBRS) [22] and *Automatic IBRS* (AutoIBRS) [23] to improve IBRS; both mitigations are *always-on* and do not require expensive model specific register writes on privilege transitions. Mitigations that *sanitize* the BPU remove potentially malicious entries before they can influence a victim in a different protection domain. The *Indirect Branch Prediction Barrier* (IBPB) allows developers to explicitly flush BPU state during domain transitions that are not otherwise sufficiently isolated. BPU state may also be shared among *sibling threads* on CPUs with *Simultaneous Multithreading* (SMT) enabled. *Single Threaded Indirect Branch Predictor* (STIBP) [24] is a mitigation that restricts BPU state sharing across SMT threads.

Available mitigations vary across microarchitectures, and the default mitigation depends on both hardware support and kernel configuration. Table 1 shows the available mitigations on the systems we evaluated and indicates whether they are enabled by default on Ubuntu 24.04.

2.5. Motivation

While Spectre-BTI attacks from user to kernel showcase a security violation, they require code execution on the victim machine which limits the impact of such attacks. Arguably, the more interesting threat model for Spectre-BTI is in the cloud where it is easy for an attacker to rent a VM and execute their attack against the hypervisor or other VMs. Previous work [1], [5] has explored the case where a malicious guest targets the hypervisor. These attacks, however, rely on modifying the code in the hypervisor to inject the desirable gadgets, which significantly limits their practicality and impact. This raises an important question: *can Spectre-BTI attacks be mounted across virtualization boundaries without imposing overly strong assumptions?*

TABLE 1. EVALUATED MICROARCHITECTURES AND THEIR ISOLATION MECHANISMS AGAINST SPECTRE-BTI.

Vendor	CPU	Year	Codename	Microarchitecture	Microcode	Isolation Mechanism			
						IBPB	IBRS	eIBRS/AutoIBRS	STIBP
Intel	Core i7-8700	2017	Coffee Lake S	Coffee Lake	0xfa	◓	●	○	◓
	Core i7-13700K	2022	Raptor Lake	Raptor Cove	0x12c	◓	○	●	●
				Gracemont	0x12c	◓	○	●	●
AMD	Ryzen 5 1600X	2017	Summit Ridge	Zen 1	0x8001137	◓	●	○	○
	EPYC 7252	2019	Rome	Zen 2	0x8301038	◓	●	○	●
	EPYC 7413	2021	Milan	Zen 3	0xa0011d3	◓	●	○	●
	Ryzen 7 7700X	2022	Raphael	Zen 4	0xa601209	◓	◓	●	●
	Ryzen 5 9600X	2024	Granite Ridge	Zen 5	0xb404023	◓	◓	●	●

○ Not available ◓ Available ● Available and Default (Ubuntu 24.04)

3. Threat Models

The goal of the attacker is to infer secrets across the host-guest virtualization boundary. In particular, we consider the following three threat models: First, $TM_{G \to H}$ considers the scenario where an attacker controlling a guest VM targets the host system. Second, $TM_{H \to G}$ represents a malicious host targeting a guest running on the system. This threat model is particularly relevant when considering guests that rely on Intel's *Trust Domain Extensions* (TDX) or AMD's *Secure Encrypted Virtualization - Secure Nested Paging* (SEV-SNP) to defend against malicious hosts. Third, instead of targeting the host, a malicious guest can target another guest, which is reflected by the $TM_{G_1 \to G_2}$ model. We assume the use of hardware virtualization extensions: SVM [10] for AMD and VT-x [14] for Intel. Furthermore, we assume the absence of software vulnerabilities in all components, in particular in the host and guest kernel, and the $\text{Hypervisor}_{\text{user}}$. Finally, we assume that the systems employ all the default security mitigations against Spectre-BTI, as listed in Table 1.

4. Overview of Challenges

Existing Spectre-BTI mitigations aim to isolate BPU state across protection domain boundaries. While the most obvious case of a malicious guest attacking the hypervisor has been explored in the past [1], [5], a systematic analysis of BPU state isolation in virtualized environments when considering the different privilege levels in the guest and host is still lacking. This brings us to our first challenge:

Challenge C1. Identifying gaps in BPU's protection domain isolation in virtualized environments.

To address this challenge, we need to carefully assess the isolation of the individual structures of the BPU. In Section 5, we systematically test the isolation that is provided by the BTB and IBP/ITA when considering different privilege levels in the guest and host. Our systematic analysis shows that many of the recent microarchitectures are susceptible to some *Virtualization-based Spectre-BTI* (vBTI) primitives, indicating a lack of proper host-guest isolation in the BPU.

However, the exact consequences and potentials for exploitation using these new vBTI primitives are not directly evident. Our next challenge is to identify the precise circumstances under which an attacker could build relevant attacks in different threat models.

Challenge C2. Understanding and identifying different threat models affected by the porous BTB isolation in a virtualized environment.

Addressing this challenge necessitates a detailed analysis of the requirements for an attacker to exploit the identified vBTI primitives. In Section 6, we present five novel attack scenarios within the three threat models introduced in Section 3. These attack scenarios allow an adversary to attack a victim across the virtualization boundary using the identified primitives. Among these scenarios, we target a novel cross-privilege vBTI primitive for exploitation, in which a malicious guest attacks the $\text{Hypervisor}_{\text{user}}$ to leak sensitive information, such as cloud infrastructure secrets. Although the underlying speculation primitive is conceptually simple, constructing a practical, end-to-end exploit remains a challenge.

Challenge C3. Enabling practical exploitation and data leakage across virtualization boundaries.

The limited interaction between the guest and host software layers makes it challenging to find suitable speculation gadgets and reliable side channels. A critical requirement for successful data leakage is ensuring a sufficiently large speculation window. However, effective cache eviction on modern AMD Zen microarchitectures, in particular on Zen 4 and Zen 5, is an open problem. In Section 7, we detail the construction of the first reliable cache evictions on AMD Zen 4 and Zen 5. Building on these findings, Section 8 develops VMSCAPE, a practical Spectre exploit that enables a guest to leak host memory across the virtualization boundary using one of our newly discovered vBTI primitives.

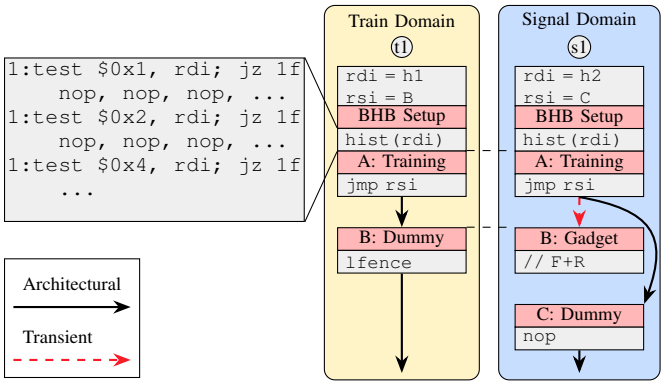


Figure 3. Training ((t1)) and signaling ((s1)) traces of the vBTI experiment allow us to evaluate BPU state isolation across the training–signaling domain boundary. The indirect branch source A and destination B are mapped to the same virtual addresses in both domains. Each indirect branch is preceded by a BHB setup, a sequence of branches that initializes the BHB to the same state in both traces using a seed value in `rdi`. This ensures that the BPU cannot distinguish between training and signaling branches, as both collide in the BTB and ITA/IBP. If isolation is lacking, the signaling branch deviates from the architectural target given by `rsi` and transiently executes the trained target at B. We detect this misprediction using a disclosure gadget.

5. BPU Isolation under Virtualization

We analyze the isolation of indirect branch prediction across protection domain boundaries in virtualized environments on a range of microarchitectures. To assess the overall isolation guarantees, we need to consider the isolation of the static and dynamic indirect branch predictors individually, and test for interference among all four fundamental protection domains of x86 with hardware virtualization. These are namely HU, HS, GU, and GS. We list all systems that we evaluate in Table 1. Recent Intel CPUs like Raptor Lake feature a heterogeneous design consisting of two microarchitectures, one optimized for high-performance, the other for low-power tasks.

5.1. Methodology

To evaluate cross-domain BPU interference for indirect branches, we use a classic Spectre-BTI [1] experiment, as illustrated in Figure 3. We train in one domain, and signal for interference in another domain, using a FLUSH+RELOAD side channel [20].

This analysis requires us to execute arbitrary code in all four evaluated domains. For HS code execution, we implement a kernel module that enables a user to run arbitrary user code as supervisor. This necessitates disabling *Supervisor Mode Execution Prevention* (SMEP) and *Supervisor Mode Access Prevention* (SMAP) on the host system. For guest execution, we rely on the KVM selftest infrastructure that is part of the Linux kernel. This simplistic hypervisor allows us to map code to arbitrary guest virtual addresses and execute it. However, KVM selftests run as GS by default, leading us to extend the framework and implement an additional privilege transition to reach GU.

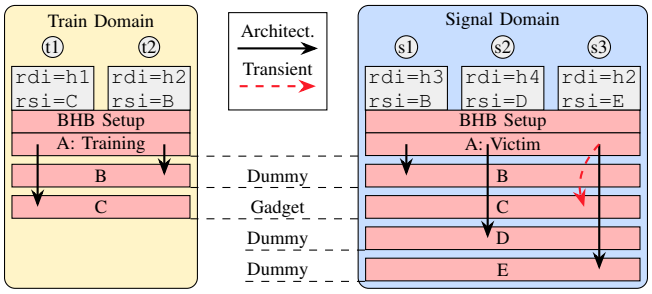


Figure 4. By combining different training traces ((t1), (t2), (s1), (s2)) we can induce the BPU to use the prediction for the signaling trace (s3) from either the BTB or the IBP/ITA. The virtual addresses of the branch source A and targets B and C are identical across both domains.

It is critical for our evaluation that we verify the isolation for all structures involved in the prediction of indirect branches. These structures include the BTB and IBP on Intel, and BTB and ITA on AMD. The reverse engineering requires us to control the state of the BHB to cause or prevent collision in the IBP/ITA. As shown in Figure 3, we initialize the BHB by executing a pseudo-random sequence of branches, depending on a control seed. This history setup snippet is placed directly preceding the training and victim branches, giving control over the BHB states at those branches.

Note that all modifications mentioned in this section are solely for the purpose of the reverse engineering. The final attack presented in Section 8 does not require any software modifications, and runs with the default mitigations, as listed in Table 1.

BTB prediction. As visualized in Figure 2, multiple structures are involved in the prediction of the indirect branches. Hence, a precise experiment setup is required to ensure that the prediction is served from the BTB and not the IBP/ITA. Intel always checks for a hit in the BTB but gives precedence to IBP hits whenever available [17]. AMD uses the BTB to predict all indirect branches that have only seen a single target, otherwise it prefers predictions from the ITA [18].

In the training domain, the execution of the training branch from A to B inserts an entry into the BTB on both Intel and AMD (Figure 3 (t1)). After switching to the signaling domain, we execute the signaling branch at A to C ((s1)). As A is the same virtual address in both the training and signaling domain, the BTB is unable to distinguish the two. Additionally, the distinct BHB states achieved by using seed values $h_1 \neq h_2$ ensure that the IBP on Intel cannot provide a prediction and falls back to the BTB. On AMD, since branch source A has so far only encountered the target C, no entry has been created in the ITA, making it get predicted by the BTB too. To ensure a clean state, we place an IBPB in-between each iteration. Getting a signal in this experiment shows susceptibility to vBTI.

IBP/ITA prediction. The key challenge of testing the dynamic predictor is that we need to ensure the predictions are not served from the BTB. This requires us to execute specific

TABLE 2. SUMMARY OF THE OBSERVED VBTI PRIMITIVES FOR ALL EVALUATED SYSTEMS. EMPTY HALVE CIRCLES INDICATE VULNERABLE, WHILE COLORED ONES INDICATE THE DEFAULT MITIGATION PREVENTING A LEAK.

Microarch.	vBTI / vBTI-SMT Primitive									
	HU→GU	HU→GS	HS→GU	HS→GS	GU→HU	GU→HS	GS→HU	GS→HS	$G_1U \to G_2U$	$G_1S \to G_2U$
Zen 1	◐	●	◐	●	◐	●	◐	●	◑	◑
Zen 2	◑	◑	◑	◑	◑	◑	◑	◑	◑	◑
Zen 3	◑	◑	◑	◑	◑	◑	◑	◑	◑	◑
Zen 4	◑	◕	◑	◕	◑	◕	◑	◕	◑	◑
Zen 5	◑	◕	◕	◕	◑	◕	◕	◕	◑	◑
Coffee Lake	◐	●	◐	●	◐	●	◐	●	◑	◑
Raptor Cove	◕	◕	◕	◕	◕	◕	◕	◕	◑	◑
Gracemont	◕	◕	◕	◕	◕	◕	◕	◕	◑	◑

same-thread is: ◐ vulnerable, or protected by: ● retpoline, ◑ IBPB, ◕ AutoIBRS/eIBRS
cross-SMT is: ◐ vulnerable, or protected by: ● retpoline, ◕ STIBP

training traces, as visualized in Figure 4. In the training domain we execute the training branch at A twice, each time to a different target. To force the BPU to distinguish the two invocations, we set different history seeds $h_1 \neq h_2$. Consequently, not only an entry in the BTB is created, but also inside the IBP/ITA.

The transition from the training to the signaling domain may invalidate or isolate BTB entries. However, on both Intel and AMD, a valid entry in the BTB is necessary to get a prediction from the IBP/ITA. Therefore, we need to ensure a corresponding BTB entry is present after switching to the signaling domain. On Intel, a single encounter of the indirect branch at A (ⓢ1) is sufficient to make subsequent encounters (ⓢ3) get predicted by the IBP. On AMD, we need an additional branch encounter (ⓢ2) to a different target, before the prediction for the final branch encounter (ⓢ3) is served from the ITA. If we get a signal on a domain transition in this experiment but not to the prior one, it indicates an imbalanced isolation of the BTB and IBP/ITA.

5.2. Primitive Evaluation

To assess the isolation guarantees, we systematically test all combinations of protection domains across the virtualization and privilege boundaries. We enable all the default mitigation mechanisms listed in Table 1. Additionally, we have repeated all the experiments once with indirect jumps and once with indirect calls as the training and signaling instructions, allowing us to assess whether both are predicted similarly and isolated properly. Our results indicate no difference between indirect jumps and indirect calls, suggesting that they are handled equivalently by the mitigations. Hence, we do not distinguish between them in the remainder of this work. Moreover, we observe that the interference signal is symmetric between the guest and the host, indicating that the isolation mechanism is equivalent in both directions. We use the notation $\text{vBTI}_{\text{X}\to\text{Y}}$ for a vBTI primitive with training in domain X and signaling in domain Y.

Table 2 shows the susceptibility of each microarchitecture to the vBTI primitives. Since we find no differences in vulnerability between the BTB and the IBP/ITA, Table 2 does not need to distinguish between them. We observe that all microarchitectures, except for Intel Raptor Cove and Gracemont, are susceptible to some vBTI primitives. Most notably, Zen 1 through Zen 5 are vulnerable to $\text{vBTI}_{\text{HU}\to\text{GU}}$ and $\text{vBTI}_{\text{GU}\to\text{HU}}$, with Zen 1 to Zen 4 additionally vulnerable to $\text{vBTI}_{\text{HS}\to\text{GU}}$ and $\text{vBTI}_{\text{GS}\to\text{HU}}$, as visualized in Figure 5. While both Raptor Cove and Gracemont are adequately protected, Coffee Lake is also vulnerable to the aforementioned primitives for Zen 1 to Zen 4. However, we find that no microarchitecture is susceptible to any primitive targeting the HS and GS domains.

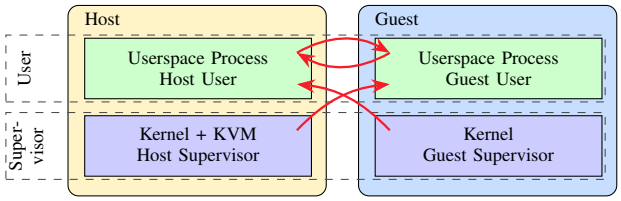


Figure 5. Effective protection domain isolation on Zen 4, susceptible to $\text{vBTI}_{\text{GU}\to\text{HU}}$, $\text{vBTI}_{\text{GS}\to\text{HU}}$, $\text{vBTI}_{\text{HU}\to\text{GU}}$, and $\text{vBTI}_{\text{HS}\to\text{GU}}$.

Observation O1. Despite the latest mitigations being enabled, all AMD Zen microarchitectures and Intel Coffee Lake are vulnerable to $\text{vBTI}_{\text{GU}\to\text{HU}}$ and $\text{vBTI}_{\text{HU}\to\text{GU}}$. Additionally, Zen 1 through Zen 4 and Coffee Lake are affected by $\text{vBTI}_{\text{GS}\to\text{HU}}$ and $\text{vBTI}_{\text{HS}\to\text{GU}}$.

BPU isolation mechanism. AMD Zen 4 and Zen 5 employ AutoIBRS as a Spectre-BTI mitigation. While in supervisor mode, it prevents the speculative execution of indirect branch targets until they are resolved [5], [6]. By rerunning the previous experiments with AutoIBRS disabled, we can gain further insights into the isolation enforcement on Zen 4 and Zen 5 microarchitectures. While Zen 4 becomes susceptible to all vBTI primitives in the absence of AutoIBRS, Zen 5 remains unaffected by primitives that cross user–supervisor boundaries. This suggests that AMD Zen 5 CPUs feature an additional native isolation mechanism on top of AutoIBRS, equivalent to a single-bit privilege level tag for prediction entries.

Observation O2. On Zen 5, branch prediction entries are tagged with a single-bit privilege level to isolate user and supervisor domains.

However, a single bit is insufficient in distinguishing between the four domains present in virtualized environments. Consequently, AutoIBRS is still required to isolate, for example, the HS and GS domains.

Observation O3. Despite the addition of isolation bits in the BTB, the BPU on Zen 5 is unable to distinguish between host and guest domains and therefore still relies on AutoIBRS to protect the supervisor domains.

SMT isolation. To complement the previous experiments that tested same-thread isolation, we test for vBTI-SMT primitives using a modified experiment. We have a training process that iteratively executes a training branch (Figure 3 (t1)). The signaling process is then bound to the SMT sibling core and executes the signaling branch with different history ((t2)). After the signaling, we flush the BPU using an IBPB to make it *forget* about the gadget destination. As listed in Table 2, while STIBP on Zen 2 through Zen 5 and Raptor Lake successfully isolates sibling threads, the lack of STIBP on Zen 1 enables the vBTI-SMT$_{\text{HU}\to\text{GU}}$, vBTI-SMT$_{\text{GU}\to\text{HU}}$, vBTI-SMT$_{\text{HS}\to\text{GU}}$, and vBTI-SMT$_{\text{GS}\to\text{HU}}$ primitives. Furthermore, we find that although Coffee Lake supports STIBP, it is not always enabled by default, thereby allowing the same cross-SMT primitives.

Observation O4. With STIBP entirely absent on Zen 1 and only conditionally enabled on Coffee Lake, both microarchitectures appear vulnerable to vBTI-SMT$_{\text{GU}\to\text{HU}}$, vBTI-SMT$_{\text{HU}\to\text{GU}}$, vBTI-SMT$_{\text{GS}\to\text{HU}}$, and vBTI-SMT$_{\text{HS}\to\text{GU}}$.

6. Exploitation in Different Threat Models

Our experiments have shown that some virtualization boundaries are not adequately isolated, allowing branch target interference across these boundaries. In this section, we contextualize our findings with respect to the three threat models introduced in Section 3, and discuss five realistic, previously unexplored scenarios in which these findings lead to concrete security violations. We represent each attack scenario as $A\to B \mid V$, where A and B denote the protection domains of the attacker and the victim, respectively, and V refers to the specific target (e.g., a host process).

6.1. Attacks from Guest to Host

The first threat model we analyze is TM$_{\text{G}\to\text{H}}$, which considers a malicious guest targeting the host system. We identify three scenarios within this model.

(S1) G→H | Host Process. In this scenario, a malicious guest targets a host user process to leak a secret, like a password hash of a SUID process [7]. The victim process may either be scheduled on the same hardware thread after the guest is preempted (i.e., *same-thread*), or on an SMT sibling thread on the same physical core (i.e., *cross-SMT thread*). In the same-thread case, in addition to the lack of guest-to-host BTB isolation, the attack also requires the absence of BTB sanitization during task switches. Based on our reverse engineering observations, we find that Zen 1 through Zen 5, as well as Intel Coffee Lake, lack sufficient isolation and are vulnerable to the vBTI$_{\text{GS}\to\text{HU}}$ and vBTI$_{\text{GU}\to\text{HU}}$ primitives. For cross-SMT attacks, guest-to-host BTB interference must be coupled with shared predictor states between sibling threads, indicated by the vBTI-SMT$_{\text{GS}\to\text{HU}}$ and vBTI-SMT$_{\text{GU}\to\text{HU}}$ primitives. This condition is met on Zen 1 and Coffee Lake microarchitectures.

While these attacks are theoretically feasible, they face several practical challenges. A guest-based attacker typically has limited influence over host user processes, making it difficult to reliably manipulate or interact with them. Establishing a robust side channel between a guest and an arbitrary host user process is particularly challenging, as the attacker lacks both spatial and temporal control. In particular, the attacker cannot dictate when or where the target process is scheduled for execution, further complicating precise exploitation.

(S2) G→H | Hypervisor$_{\text{user}}$. This scenario considers a malicious guest targeting its own Hypervisor$_{\text{user}}$. This requires insufficient BTB sanitization on `VMEXIT`s and `KVM_EXIT`s, alongside a lack of BTB state isolation between the guest and the host on the same-thread case. Compared to general G→H | Host Process attacks, exploitation here is simpler. The guest can deliberately trigger `VMEXIT`s that are handled by its Hypervisor$_{\text{user}}$, enabling frequent attacker–victim interactions in the same-SMT case. The cross-SMT case is also more practical. Ordinarily, scheduling mitigates cross-SMT BTI attacks by reducing the likelihood of colocating attacker and victim. Here, however, the victim executes the Hypervisor$_{\text{user}}$ of the same guest, making colocation much more likely. Although the attacker can only access memory mapped into the Hypervisor$_{\text{user}}$'s address space, this can still include sensitive data, such as cloud infrastructure secrets used for networking or storage [25]. We observe that Zen 1 through Zen 5 and Coffee Lake are vulnerable to this scenario, with Zen 1 and Coffee Lake additionally being vulnerable to the cross-SMT case.

Observation O5. Cross-SMT G→H | Hypervisor$_{\text{user}}$ attacks are facilitated as the attacker and victim belong to the same guest execution, making colocation far more likely.

(S3) G→H | Guest Kernel. In this scenario, the attacker leverages the Hypervisor$_{\text{user}}$ as a confused deputy to extract kernel memory from the attacker's own guest instance. As such, this scenario assumes that the attacker is an unprivileged process running in the guest. It is a variant of the G→H | Hypervisor$_{\text{user}}$ scenario, but instead of leaking a Hypervisor$_{\text{user}}$ secret, it leaks the guest's own memory, that is being mapped in the Hypervisor$_{\text{user}}$. Besides the requirement for the Hypervisor$_{\text{user}}$ to have the guest's memory mapped, the prerequisites are identical to G→H | Hypervisor$_{\text{user}}$. Specifically, these prerequisites include the lack of guest-to-host BTB isolation and inadequate sanitization on `VMEXIT`s and `KVM_EXIT`s.

Observation O6. Multiple recent microarchitectures, including Zen 5, are vulnerable to G→H | Guest Kernel attacks.

TABLE 3. ATTACK SCENARIOS BY THREAT MODELS, INDICATING REQUIREMENTS, TARGET, AND VULNERABLE MICROARCHITECTURES.

Scenario	Target	SMT	Required Primitives	Exploitable on: Zen 1	Zen 2	Zen 3	Zen 4	Zen 5	Coffee Lake	Raptor Cove	Gracemont
(S1) G→H | Host Process	Host user process	Same	$vBTI_{GS\to HU}$ or $vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GS\to HU}$ or $vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S2) G→H | $Hypervisor_{user}$	$Hypervisor_{user}$	Same	$vBTI_{GS\to HU}$ or $vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GS\to HU}$ or $vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S3) G→H | Guest Kernel	Guest supervisor	Same	$vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S4) H→G | SEV-SNP Process	Guest user process	Same	$vBTI_{HU\to GU}$ or $vBTI_{HS\to GU}$	-	-	✓	?	✗	-	✗	✗
		Cross	$vBTI\text{-}SMT_{HU\to GU}$ or $vBTI\text{-}SMT_{HS\to GU}$	-	-	✗	✗	✗	-	✗	✗
(S5) $G_1\to G_2$ | Guest Process	Other guest user process	Same	$vBTI_{G_1U\to G_2U}$	✗	✗	✗	✗	✗	✗	✗	✗
		Cross	$vBTI\text{-}SMT_{G_1U\to G_2U}$	✓	✗	✗	✗	✗	✓	✗	✗

✓ Vulnerable ✗ Not vulnerable - Not applicable ? Not supported on the tested system

6.2. Attacks from Host to Guest

The second threat model we analyze is $TM_{H\to G}$ in which a malicious host targets a guest.

(S4) H→G | SEV-SNP Process. In this scenario, a malicious host targets a guest, typically one that assumes a hostile host and employs a hardware-based isolation technology such as SEV-SNP [10] or TDX [11]. Unlike in $TM_{G\to H}$, the attacker controls the scheduling of the guest thread, providing the attacker more control over the victim. Since these technologies impose additional security measures, the presence of working $vBTI_{HS\to GU}$, $vBTI_{HU\to GU}$, $vBTI\text{-}SMT_{HS\to GU}$, and $vBTI\text{-}SMT_{HU\to GU}$ primitives on systems without SEV-SNP/TDX is not sufficient to determine vulnerability of different microarchitectures. However, we were able to confirm that on Zen 3, by default, SEV-SNP does not isolate the BTB between the guest and the host. On Zen 5, we noticed that the predictions appear properly isolated between the host and an SEV-SNP guest. Unfortunately, we do not possess a Zen 4 server to check this isolation. While Zen 1 and Zen 2 do not support SEV-SNP, they could be vulnerable in scenarios using weaker technologies like SEV or SEV-ES.

Observation O7. AMD Zen 3 is vulnerable to H→G | SEV-SNP attacks.

SEV-SNP provides various optional protection mechanisms for guests, which we will discuss in Section 9.3. While our analysis demonstrates the susceptibility of older, pre-eIBRS Intel microarchitectures to $vBTI_{HS\to GU}$, $vBTI_{HU\to GU}$, $vBTI\text{-}SMT_{HS\to GU}$, and $vBTI\text{-}SMT_{HU\to GU}$ primitives, these systems do not support TDX.

6.3. Attacks Between Guests

The third threat model we analyze is $TM_{G_1\to G_2}$. In this model, a malicious guest attacks another guest running on the same host system.

(S5) $G_1\to G_2$ | Guest Process. In this scenario, a malicious guest targets a user process of another guest, highlighting the risk of co-residency between untrusted VMs. This requires a lack of BTB isolation between different guests. As with earlier scenarios, we differentiate between a same-thread and cross-SMT case. In the same-thread case, the victim executes on the same thread as the attacker, following a preemption of the attacker's thread. This additionally requires the lack of BTB sanitization on VM switches. However, this is mitigated in the Linux kernel through an IBPB on VM switches. The cross-SMT case, instead, requires that BTB state be shared across sibling threads. Based on our reverse engineering in Section 5.2, we find that both Zen 1 and Coffee Lake lack SMT thread isolation, rendering them vulnerable to $G_1\to G_2$ | Guest Process attacks.

Observation O8. AMD Zen 1 and Intel Coffee Lake are vulnerable to $G_1\to G_2$ | Guest Process attacks.

Practical exploitation, however, remains challenging. There is generally no direct channel for interactions between an attacker and guest VM and shared memory for a side channel on public cloud hosts.

6.4. Summary

Table 3 provides an overview of the attack scenarios, summarizing their prerequisites, intended targets, and the vulnerable microarchitectures. Our analysis demonstrates that vBTI primitives can lead to concrete security violations across all three threat models introduced in Section 3. These include attacks from a malicious guest against the host ($TM_{G\to H}$), from a malicious host against a guest ($TM_{H\to G}$), and even between two guests ($TM_{G_1\to G_2}$).

To demonstrate the feasibility of vBTI attacks, we present VMSCAPE, an end-to-end exploit targeting the G→H | $Hypervisor_{user}$ scenario. This scenario represents

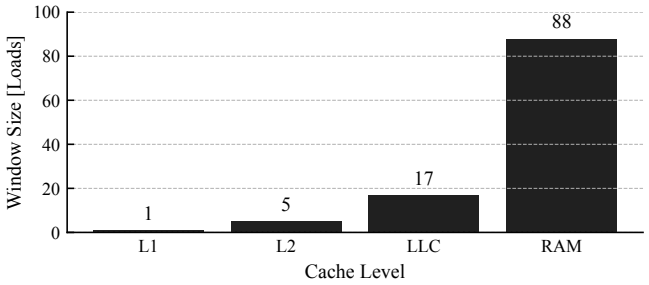


Figure 6. Speculation window size on Zen 4 as the maximum number of executed dependent loads with the pointer to the indirect branch destination residing in the respective cache level.

a particularly severe case for cloud providers, as leaking $\text{Hypervisor}_{\text{user}}$ memory targets their infrastructure.

7. Increasing the Speculation Window

Our aim is to build an end-to-end memory leak exploit in the G→H | $\text{Hypervisor}_{\text{user}}$ scenario, targeting up-to-date AMD Zen 4 and Zen 5 systems. While building this exploit, we observed that the speculation window of the victim branch was too short. We first explain how we assess the speculation window size and then discuss how we can enlarge it using cache eviction.

Speculation Window Size. To measure the size of the speculation window, we design the following experiment: we use an indirect branch that takes its target address from memory, resulting in the BPU predicting and speculating to the target. The size of the speculation window depends on how quickly the target address can resolve architecturally, which in turn depends on the cache level holding the target address of the indirect branch. We can measure the size of the speculation window by chaining n dependent loads during speculation and checking for the cache hit of the last load. We ensure that all loads, except for the last one, reside in the L1 cache. We repeat the experiment 4096 times for each $n \in [1, 127]$, and vary the cache level holding the target of the indirect branch.

As shown in Figure 6, when the target address resides in L1, only a single dependent load is executed during the speculation window. Such a speculation window is insufficient for a FLUSH+RELOAD-based side channel, since it requires at least two dependant loads. While our results for L2 indicate a sufficiently large speculation window for a perfect speculation gadget, attacks sometimes require even larger speculation windows. This can be due to additional contention from previous instructions that slow down the disclosure gadget, or due to the victim software starting to load the target address already earlier in the instruction stream (e.g., to check if it is a valid pointer.). Such effects reduce the speculation time available to the disclosure gadget, requiring the attacker to extend the overall speculation window size.

The speculation window can be extended by evicting the memory that holds the destination of the indirection branch from the cache, thereby slowing the resolution of the victim branch [1]. However, an attacker inside a VM cannot simply flush the target pointer as the physical memory that backs it is allocated to the host process and not available to the VM. Hence, the attacker needs to evict the target pointer from the cache hierarchy instead. We note that eviction should target only the necessary cache sets to avoid slowing down the entire victim execution during speculation.

TABLE 4. CACHE DETAILS ON AMD ZEN 3, ZEN 4 AND ZEN 5. THE SIZES MAY VARY BETWEEN PROCESSORS OF THE SAME GENERATION.

Core	L1			L2			*Last Level Cache* (LLC)		
	Size	Sets	Ways	Size	Sets	Ways	Size	Sets	Ways
Zen 3	32KiB	64	8	512KiB	2048	8	32MiB	32768	16
Zen 4	32KiB	64	8	1MiB	2048	8	32MiB	32768	16
Zen 5	48KiB	64	12	1MiB	1024	16	32MiB	32768	16

7.1. LLC eviction on AMD Zen 4 and Zen 5

Recent work by Wang et al. [26] demonstrated the feasibility of creating LLC eviction sets on AMD Zen 3. We build on their insights to present the first LLC evictions for AMD Zen 4 and Zen 5. To achieve this goal, we progress through all levels of the cache hierarchy, identifying differences to Zen 3 and learning to evict progressively larger cache levels, which results in increasingly longer speculation windows. We then show how the eviction set building can be ported inside a VM and how the XOR-based indexing can be exploited to make the process highly efficient and reliable. Table 4 lists relevant cache geometries for the evaluated processors. In the interest of brevity, we first write exclusively about Zen 4 before summarizing the differences to Zen 5.

For reverse engineering purposes, we use 1 GB paging to ensure matching bits between virtual and physical addresses. We further use AMD's accurate `APERF` timer, which is accessible through the `rdpru` instruction [27]. By default, both the 1 GB paging as well as the precise counter are not available to guests. Consequently, we only use these features during reverse engineering and not for the end-to-end exploit in Section 8.

Cache access latencies. We first calibrate the memory access timing function for each cache level. This allows us to determine whether a memory access was served from L1, L2, LLC, or main memory. To determine the latencies, we proceed as follows. We start with a memory region consisting of N cache lines. We access all cache lines in pseudorandom order, recording the access times. We repeat this procedure for increasing values of N. In Figure 7, we plot the median access time for increasing memory region size N. The latency plateaus indicate the L1, L2, LLC, and RAM access latencies.

L1 eviction sets. Based on our earlier cache access measurements, we can confirm that the L1 indexing scheme on Zen 4 matches Zen 3. We can evict any L1 entry by accessing 8 distinct cache lines (corresponding to the L1 ways), all of which share matching bits $[6:11]$.

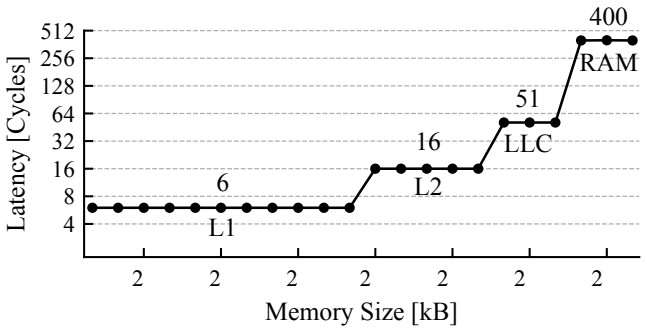


Figure 7. Median memory access latencies for a memory region of increasing size on Zen 4, shown on logarithmic axes. The latency readings of the plateaus indicate the L1, L2, LLC, and RAM latencies.

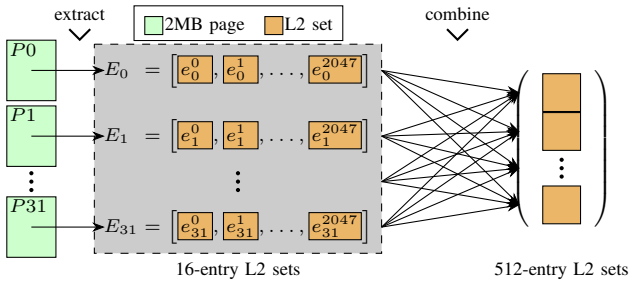


Figure 8. LLC cache eviction set building strategy for VM-based attackers. In a first step, we *extract* the L2 eviction sets contained in different 2 MB pages. In a second step, we then *combine* L2 sets originating from different 2 MB pages.

L2 eviction sets. Let us assume that L2 indexing is designed in a way such that the memory inside a 2 MB huge page is evenly distributed across all L2 sets. Consequently, at least 11 indexing bits below the 2 MB page boundary are required to map to all 2048 L2 cache sets. Based on prior work on cache reverse engineering of Zen 3 [26], we further assume that some higher bits below the 2 MB page boundary are not used for L2 indexing. Given that there are 21 address bits in a 2 MB page where 11 bits are used for indexing and 6 bits are used for the offset within a 64 B cache line, we hypothesize that the 4 highest bits are not used for cache indexing. This would result in 16 entries per L2 set which we expect to be sufficient for eviction of all 8 ways.

We verify the above hypothesis as follows. We select a random 2 MB huge page and split all cache line-aligned addresses within the huge page into sets according to the value of bits $[6:16]$. We then confirm that each of the generated sets is self-evicting by measuring the mean latency of repeatedly accessing all entries within a set. We further confirm that no two generated sets are mutually evicting by measuring the latency of repeatedly accessing half the addresses from each set. Given our results, we conclude that the L2 cache indexing on AMD Zen 4 does not use bits $[17:20]$.

Observation O9. Bits $[17:20]$ remain unused in L2 cache indexing on AMD Zen 4.

The L2 on AMD Zen 4 has 8 ways which leads us to expect a minimal eviction set size of 8 addresses, given a *Least Recently Used* (LRU) replacement policy. However, we find that at least 12 eviction addresses are required to reliably evict a victim address from the L2 cache in our experiments. We attribute this behavior to a more complex replacement policy than LRU.

LLC eviction sets. The LLC on AMD Zen 4 is a non-inclusive victim cache for the L2. Hence, to insert cache lines into the LLC for eviction, said cache lines need to first be evicted from the L2. Given an L2 eviction set, we can identify all addresses within the same 1 GB huge page that map to this eviction set by testing whether they are evicted by it. Assuming an even distribution of addresses across the 2048 L2 sets within a 1 GB huge page, we would expect 8192 addresses to map to the same L2 set, which our experiments confirm. We have further confirmed that the first 512 of these addresses already provide reliable LLC eviction for any address within the same L2 cache set. While this does not constitute a minimal eviction set, it is sufficiently efficient for our attack and significantly increases the length of the speculation window, as shown in Figure 6 (RAM).

Zen 5. Due to the additional ways in the L1 and L2 caches, the cache eviction set sizes increase correspondingly. Zen 5 reduced the number of L2 sets compared to Zen 4 to again match Zen 3. Hence, we find that the same bits within the 2 MB pages remain unused as in Zen 3, namely $[16:20]$. Lastly, we require twice the number of addresses and have to access each address twice to reliably increase the speculation window on Zen 5 when compared to Zen 4.

7.2. Eviction from a VM

Our LLC eviction enables our attack to succeed, but it is using the precise counter and 1 GB paging which are not available to QEMU guests by default. Furthermore, increasing the size of each L2 eviction set by brute-forcing the set membership of new addresses is slow and susceptible to noise. First, we overcome the timer limitation by measuring the access times of all entries in an eviction set together rather than separately to amplify the signal. Second, we propose a significantly more efficient eviction set generation approach without relying on 1 GB huge pages.

We note that Linux, by default, aims to back VM pages with transparent 2 MB huge pages to improve performance. Hence, our approach for constructing L2 sets using 2 MB pages remains applicable from within a VM. Moreover, we argue that given many such 2 MB pages, we can *extract* L2 sets for each of them as shown in Figure 8. While this leaves us with sufficient addresses for LLC eviction, we need to find an efficient approach to *combine* L2 sets generated from different pages.

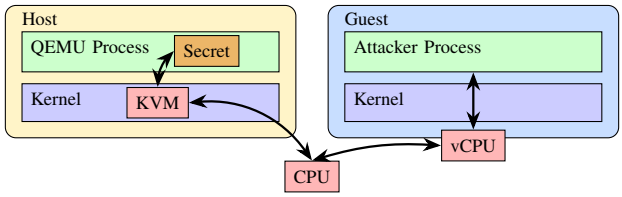


Figure 9. KVM-based guest running on a CPU with hardware virtualization support, managed by QEMU.

Let e_j^i be the i-th eviction set originating from page j where i is given by the lower L2 set indexing bits $[6:16]$. Let $E_j = \{e_j^i | i \in [0, 2047]\}$ be the set of eviction sets for page j. Our aim is to merge all L2 sets in E_j $(j \neq 0)$ into L2 sets of E_0. Now, assume that for some j, a, and b, the set e_j^a collides with the set e_0^b. We would like to efficiently determine with which set e_j^c will collide. We hypothesize that the L2 and LLC are using XOR-based indexing functions which would allow calculating the colliding set e_0^d as follows:

$$d = a \oplus b \oplus c$$

Using this approach, finding the matching L2 set for a single element of each E_j $(j \neq 0)$ is sufficient to determine the colliding eviction sets for all other elements. The previous approach on Zen 3 [26] would increase the size of the initial L2 sets by sampling addresses one by one and testing *each* for eviction. Our new approach only requires this process *once*, using the whole first L2 set within a 2 MB page, which then allows all other 32752 addresses from the same page to be assigned their respective set without further testing. Additionally, the approach is highly reliable, creating LLC eviction sets with 100% success rate on the first try over 100 runs whereas previous work [26] reported around 60% on Zen 3 when using 1 GB pages.

Observation O10. XOR-based cache indexing functions enable efficient merging of L2 sets from different 2 MB pages into bigger L2 sets.

With the successful extension of the speculation window, we now proceed to construct VMSCAPE.

8. VMSCAPE

Our cross-domain experiments introduced in Section 5 unveiled that all AMD Zen CPUs and Intel Coffee Lake CPUs are vulnerable to the $\text{vBTI}_{\text{GU}\rightarrow\text{HU}}$ primitive. In this section, we present VMSCAPE, our end-to-end exploit to leak hypervisor secrets as a malicious guest. Figure 9 illustrates the involved components. We target QEMU, a common $\text{Hypervisor}_{\text{user}}$ in the Linux ecosystem used in industry [15]. However, none of our assumptions in the threat model (see Section 3) or in the design of the exploit fundamentally prevent targeting a different $\text{Hypervisor}_{\text{user}}$ with a slightly modified version of the same technique. We run the attack on an AMD Zen 5 server CPU with Linux kernel version 6.8.0-85-generic and Ubuntu 25.10 packages, featuring the recent QEMU v10.0.2. We further verified that VMSCAPE leaks information successfully on an AMD Zen 4 system.

In Section 7, we addressed the speculation window challenge of the attack. However, to develop an end-to-end exploit, two additional challenges must be addressed:

- First, we need to identify a suitable exploit chain consisting of a side channel, a speculation gadget and a disclosure gadget (Section 8.1).
- Second, we need to derandomize the *Address Space Layout Randomization* (ASLR) of QEMU to find the victim branch address as well as the reload buffer address (Section 8.2).

Finally, we combine our solutions with the speculation window lengthening technique to construct an arbitrary memory leak attack against QEMU (Section 8.3).

8.1. Exploit chain

We aim to find an exploit chain in unmodified code of QEMU. To construct such a chain, we need to find a suitable side channel as well as a speculation gadget and a disclosure gadget in the QEMU binary.

Side channel. We begin with defining the side channel to be used for leaking host secrets. Through our investigation, we realized that QEMU maps all guest memory into its virtual address space. Shared memory between host and guest enables FLUSH+RELOAD as a side channel [20].

Observation O11. QEMU maps all guest memory into the $\text{Hypervisor}_{\text{user}}$ virtual address space, facilitating FLUSH+RELOAD.

FLUSH+RELOAD leaks secret information by detecting what memory locations (shared with the victim) were recently accessed. The shared memory that serves as the side channel is called a *reload buffer*.

Speculation gadget. Our exploit chain needs to trigger speculation for which our vBTI primitives use indirect calls or indirect jumps. We search for such speculation gadgets in places where we have some degree of control over registers in order to influence execution at the speculated target. Using FLUSH+RELOAD, we aim to control two registers, one containing a secret pointer and another containing the reload buffer address. Furthermore, a desired speculation gadget can be triggered repeatedly, reliably, and with high frequency, enabling high bandwidth leakage.

QEMU primarily handles I/O and device emulation when managing a KVM-based VM. Consequently, the active code footprint of QEMU is relatively small. We restrict our gadget search to general I/O handling code and avoid targeting specific device emulations, so as not to impose additional assumptions on the threat model.

We identified a suitable victim branch in QEMU's *Memory Mapped I/O* (MMIO) write handling as shown

```
1  static MemTxResult memory_region_write_accessor(
2                          MemoryRegion *mr,
3                          hwaddr addr,
4                          uint64_t *value,
5                          unsigned size,
6                          signed shift,
7                          uint64_t mask,
8                          MemTxAttrs attrs) {
9      uint64_t tmp = memory_region_shift_write_access(
10                        value, shift, mask);
11     // [...]
12     mr->ops->write(mr->opaque, addr, tmp, size);
13     return MEMTX_OK;
14 }
```

Listing 1. Extract from QEMU's `system/memory.c` source file, showing the victim branch on line 12, with the attacker-controlled value in `tmp`.

in Listing 1. The attacker has control over the value in `tmp` which corresponds to the value written in the MMIO operation. While the adversary only directly controls a single 64 bit register (*RDX*) at the function call, dynamic analysis revealed that the same value is also left over from earlier use in a second register (*R12*). Furthermore, we can pass additional values through the shared memory, at the cost of requiring an extra load in the disclosure gadget.

Observation O12. MMIO provides repeatable, reliable, and high-frequency interaction with register control between the guest and the hypervisor.

One caveat is that while general MMIO supports 8 B writes, the device to which we write must also support writes at this granularity. For our main exploit, we use the `hpet` high precision event timer that is present in QEMU VMs by default.

Disclosure gadget. To leak secrets from the guest, we require a disclosure gadget that leverages FLUSH+RELOAD. We build on the disclosure gadget scanner from Wikner and Razavi [2]. Since the original scanner does not find the gadgets we require in the QEMU binary, we expand the scanner by improving the tainting logic for memory instructions that use differently tainted base and index registers. Additionally, we add the capability to search for chained gadgets where we combine two separate gadgets, the first of which ends in an indirect call that we can train to mispredict to the second gadget. These changes enable us to find the desired disclosure gadget. Listing 2 shows the final gadget chain used in our attack.

The gadget does not apply any multiplier to the secret which can be problematic for FLUSH+RELOAD. However, previous work demonstrated how we can rely on a technique of shifting the base of the reload buffer and checking when the secret ends up in a different cache line to overcome this limitation [2].

Branch collisions. When the victim branch in QEMU is reached, the control flow has just passed through a `VMEXIT` followed by a `KVM_EXIT`. Thus, the branch history is influenced by branches in the guest, KVM, and QEMU,

```
1 // first part of the gadget chain
2 0xac334e mov rdi, qword ptr [r12 + 0x2b58]
3 0xac3356 call qword ptr [r12 + 0x2b50]
4
5 // second part of the gadget chain
6 0x8260b5 add cl, byte ptr [rdi]
7 0x8260b7 test dword ptr [rdx + rcx], esp
```

Listing 2. Disclosure gadget in two parts. Line 2 loads the secret pointer, line 6 retrieves one byte of the secret and line 7 accesses the reload buffer.

depending on the size of the history buffer. Since AMD's history-based branch predictor takes precedence over the static BTB predictions, this necessitates either overwriting or invalidating the history-based predictions for our injected targets to be observed. Accurately replicating the history during training is possible but challenging. According to AMD's documentation [18], [28], history is only considered for indirect branches that have encountered multiple targets. By flushing the BPU using an IBPB before training, the malicious guest can ensure that the victim branch has only observed the training target. Hence, the branch is mispredicted without requiring matching history beyond the last two branches, which are relatively easy to replicate. While guests having access to IBPB may appear to be a strong assumption, this is in fact expected for guests to be able to enforce domain isolation themselves.

Observation O13. Guest software can exploit access to mitigation controls to influence what branch prediction entries are available to host software.

However, Zen 5 and later microcode updates on AMD Zen 4 modify IBPB to also invalidate direct branch type information in the BTB in order to help mitigate *Speculative Return Stack Overflow* (SRSO) [6], [29]. This inadvertently reduced the speculation window size for our attack, presumably because direct branches between the first victim pointer load and the victim branch would now slow down the speculative execution. Hence, we have to make the BTB forget that the victim branch is an indirect branch with multiple targets without using IBPB. We can do so on AMD Zen 4 and Zen 5 by replacing the victim BTB entry with a bogus direct branch prediction as follows. (1) We replace the training branch with a direct jump instruction, (2) we execute the training procedure, and (3) we write back the original training branch. This is in line with AMD's Jmp2Ret mitigation for the Retbleed vulnerability [30].

We can now reliably and repeatedly hijack the victim branch in QEMU from the guest, redirecting transient execution to our disclosure gadget.

8.2. Breaking ASLR

Linux randomizes the virtual addresses of userspace code, heap and stack in memory using ASLR by default. To mount a successful attack, we need to know the location of the speculation gadget and the disclosure gadget as well

as the virtual address at which QEMU mapped the VM memory where our reload buffer resides.

Partial code ASLR derandomization. Linux implements ASLR as a random offset from the non-ASLR base address of a program. The maximum offset is a configurable parameter that defaults to some dynamic value, chosen by Linux. The target system uses a default maximum offset of 2^{32}.

Similar to prior work, we aim to find the address of the speculation gadget by checking all possible locations of the victim branch [8], [31]. They train a branch B_i at some hypothesized victim location V_i in the attacker context, trigger the execution of the victim and measure whether the prediction for B_i was evicted. This approach is not applicable in our case for two main reasons. First, in contrast to Intel, the BTB on AMD Zen 4 and Zen 5 does not overwrite the target with every misprediction but rather inserts the new target into the ITA while keeping the previous target in the BTB. Second, we would need to separately check up to around 2^{32} locations, which takes prohibitively long.

We optimize the above approach by reversing the attacker and victim roles as proposed by prior work [7]. Given a hypothesized victim branch location V_i, we can calculate the corresponding target address T_i. Thus, we can first trigger the victim to train the branch predictor and then we execute a matching branch in the attacker domain at location V_i to some non-signaling target. By additionally placing a signaling gadget in the attacker domain at the hypothesized victim target location T_i, we can detect collisions. Using our new approach, a single execution of the victim branch and a single reload of the reload buffer for signaling suffice to check many potential victim locations (e.g., 1024 locations). Once a signal is observed, we can determine which of the locations was correct.

Results. While this approach provides us with single victim location on Zen 4—directly breaking code ASLR, on Zen 5 we get a multitude of candidates. Because all candidates share the same lower 24 bits, we conclude that this is due to aliasing and the fact that the BTB provides partial targets, similar to Intel [8], [17]. While this reveals the lower 24 bits of the victim branch, it only partially derandomizes code ASLR which randomizes 32 bits above the page offset bits. We find than when training the predictor with only matching lower 24 bits, it also only provides the lower 24 bits of the target. Consequently, having only partially derandomized code ASLR, we can only speculate to targets within an offset of 24 bits from the victim branch. However, our selected disclosure gadget chain resides at a larger distance (than 2^{24} from the victim branch), requiring us to fully break code ASLR.

Locating the reload buffer. Before we can proceed to fully breaking code ASLR, we need to identify the virtual address where QEMU has mapped the memory used by the attacker's VM to back the reload buffer. The reload buffer we also need to eventually leak arbitrary data through FLUSH+RELOAD. Similar to prior work [2], we brute force the location of the reload buffer. The partial derandomization from before allows us to speculate to targets residing within a 24 bit offset from the victim branch. Within this region, we locate a gadget that loads the memory at the address provided by the attacker. Conveniently, as previously observed, Linux and QEMU collaborate to map VM pages as 2 MB huge pages. Hence, by mapping the reload buffer as a 2 MB huge page inside the VM, we can significantly reduce the number of potential reload buffer locations. Additionally, we find that since searching for the reload buffer only requires a single load in the speculation window. This means that we do need to lengthen the speculation window for this step.

Results. Over 100 repetitions, we can derandomize the reload buffer location with a success rate of 100% and within median 46 s and 83 s on Zen 5 and Zen 4, respectively.

Fully breaking code ASLR. Having located the reload buffer, we can now fully derandomize code ASLR and locate the speculation getdget. This also reveals the location of the disclosure gadget, since ASLR only randomizes the base address of the executable and preserves offsets. This extra step is not required for Zen 4 as the BTB did not seem to exhibit aliasing, directly yielding the victim location.

At the victim branch, the register *RAX* contains a pointer to some code location. Knowing this value would allow us to calculate the full victim branch location. We can leak this pointer using an additional gadget that is within the 24 bit offset of the victim branch. This gadget adds the attacker controlled register *RDX* to *RAX* and loads the resulting address. We leak *RAX* by brute forcing values for *RDX*, such that we register a hit in our reload buffer.

Results. Fully breaking code ASLR takes a median 12 s and 75% accuracy on Zen 5, neglecting the time to locate the reload buffer. On Zen 4, where the first step already revealed the location of the victim branch, breaking ASLR takes 238 s and has 70% accuracy. The difference in time comes from the fact that in the first step of code ASLR breaking, we only need to find a single candidate within 2^{12} possibilities in on Zen 5, while on Zen 4, finding the correct address requires checking all 2^{32} possible locations. The larger search space for Zen 4 in the first step takes significantly longer than the required second step for Zen 5 where we leak the remaining 20 bits of ASLR entropy.

8.3. Arbitrary Memory Leak

We now have all the components required for building an arbitrary memory leak attack: the exploit chain, an ASLR-breaking primitive, and a reliable approach to provide sufficiently large speculation windows.

Our VMSCAPE attack proceeds as follows. First, we locate the victim branch using our ASLR-breaking technique. Once the gadgets are identified, we can determine the QEMU virtual address of our reload buffer. Next, we construct our LLC eviction sets and identify the correct eviction set by testing which one enables dependent loads to leave a cache trace. We then use our gadgets with the long speculation window to leak QEMU's internal data structures

to find the address of the secret data and then leak said secret data.

Results. We need a median 102 s for the full end-to-end attack on Zen 5, leaking a $4KiB$ QEMU secret object, which we set to be a disk encryption and decryption key as an example. Of the total attack time, 58 s are needed to find the victim branch and the reload buffer. Once obtained, our exploit leaks arbitrary QEMU memory at a rate of 154 B/s with 99.85% byte accuracy. The overall success rate of the attack is 68%.

9. Discussion

We discuss NUMA considerations for the VMSCAPE attack (Section 9.1), additional vBTI variants that we discovered after our initial responsible disclosure (Section 9.2) and the mitigation strategies for vBTI primitives (Section 9.3).

9.1. NUMA considerations

We noticed that VMSCAPE cannot leak certain memory pages on our Zen 5 system that employs NUMA. Tracking this issue down, it turns out that the Linux NUMA balancing mechanism [32] periodically marks pages as not present to track whether they are most frequently accessed by CPU cores in the same NUMA node. Pages that are marked as not present cannot be accessed in speculation—effectively stopping VMSCAPE on memory pages that are not regularly accessed by the hypervisor. However, cloud virtualization platforms and providers typically allocate physical memory for VMs on the same NUMA node to improve performance [33], [34], [35]. In such practical settings, the Linux kernel recommends turning off NUMA balancing to reduce the unnecessary tracking overhead [32].

9.2. BHI variants

Intel's response to classical Spectre-BTI [1] was eIBRS, a hardware mitigation that effectively isolates user and supervisor entries in the BTB. *Branch History Injection* (BHI) [3], however, re-enabled cross-privilege Spectre attacks by exploiting the lack of BHB isolation between the user and supervisor domains. As we show in Section 5, eIBRS successfully enforces BTB isolation in virtualized environments. Yet, akin to BHI, eIBRS does not inherently prevent a guest from (partially) controlling the branch history. While this case is mitigated for attacks against the HS [5], [36], we believe that there is no mitigation for the HU. While we did not verify this, Intel has confirmed that this is indeed the case, indicating that recent Intel CPUs are potentially vulnerable to *Virtualization-based Spectre-BHI* (vBHI) primitives.

9.3. Mitigation

As described in Section 5, our reverse engineering revealed multiple vBTI primitives across recent microarchitectures. In this section, we examine mitigation strategies to prevent such primitives, beginning with IBPB-on-`VMEXIT`, which forms the basis of the deployed VMSCAPE patches.

IBPB-on-`VMEXIT`. This mitigation protects the host in the $\text{TM}_{\text{G}\to\text{H}}$ model by flushing the BPU state with an IBPB on every `VMEXIT`. As a result, it enforces BTB isolation between guest and host on each guest-to-host transition. However, IBPB-based mitigations typically impose substantial performance overhead [2], [6], which becomes particularly problematic when applied to a fast path such as `VMEXIT`.

To quantify this overhead, we benchmarked the mitigation using the UnixBench test suite [37], following the methodology of prior work [2], [6], [8]. We run the workloads in a guest with 4 GB of memory and two cores, in multithreaded mode. The geometric mean of the performance overhead (over five repetitions) is 57 % on Zen 5.

Linux kernel developers based their mitigation on IBPB-on-`VMEXIT`, but optimized it by conditionally issuing the IBPB only on `KVM_EXIT`s that are followed by guest execution. Since not every `VMEXIT` results in a `KVM_EXIT` and subsequent userspace execution, this change moves the IBPB off the fast path while preserving its security guarantees. They refer to this mitigation as *"IBPB before exit to userspace"*. This patch gets applied to all affected systems, including AMD Zen 5 and recent Intel CPUs such as Lunar Lake and Granite Rapids.

In addition to verifying the effectiveness of the patches, we benchmarked their performance overhead on the same setup. UnixBench shows only a marginal overhead of 1 % on Zen 4. However, as a compute-oriented benchmark, UnixBench causes relatively few exits to userspace. To evaluate a workload that causes frequent exits to userspace, we used `fio` [38] to generate disk I/O, by randomly reading and writing 10 GB on a `virtio` disk repeatedly for 10 times. On Zen 4, the optimized mitigation has an overhead of 51 %, approaching the 57 % overhead of IBPB-on-`KVM_EXIT`. When using devices handled by the host kernel (`vhost`) or a pass-through devices, exits to userspace are minimized. These results demonstrate that the performance impact of the mitigation strongly depends on both the workload and the QEMU configuration, but is expected to remain marginal in most real-world settings.

Retpoline. Retpoline is a classic software-based Spectre-BTI mitigation that replaces certain indirect branches (`jmp` and `call`) by other indirect branches (`ret`) that are predicted using a different predictor [39]. Compiling $\text{Hypervisor}_{\text{user}}$s using retpoline would protect them against attackers in the G→H | $\text{Hypervisor}_{\text{user}}$ scenario on systems not also vulnerable to earlier attacks against returns [6]. While not protecting other host user processes considering G→H | Host Process, the induced performance overhead is lower compared to IBPB-on-`VMEXIT`. Performance evaluation of retpoline using the UnixBench workloads shows an overhead of 3.9 %. Combining retpoline with selective host process isolation using the `PR_SET_SPECULATION_CTRL prctl` can serve as an alternative strategy to the high-overhead IBPB-on-`VMEXIT`, depending on the threat model. Our investigations show

that none of the popular open-source Hypervisor$_{\text{user}}$s deploy retpoline [13], [40], [41], [42]. This mitigations is only applicable to CPUs that are unaffected by Phantom speculation [43], like Zen 5.

Protecing SEV-SNP Guest. While Intel systems affected by vBTI primitives do not support TDX, SVM on Zen 3 through Zen 5 provides several optional protections for an SEV-SNP-enabled guest [18], [28]. (1) *Branch Target Buffer Isolation* restricts execution outside the guest domain from influencing branches within the guest execution. (2) *Indirect Branch Prediction Barrier on Entry* forces the CPU to issue an IBPB on every `VMRUN`. (3) *SMT Protection* forces the sibling SMT thread to be idle while the guest is running on the other sibling thread. Our experiments in Section 5 indicated that Zen 5 already provides sufficient isolation for SEV-SNP guests by default. Hence, such mitigations are appropriate on earlier CPUs.

Protecting SMT. STIBP is a hardware mitigation that isolates the BPU of SMT cores. We identified that most systems enable STIBP by default, protecting against vBTI-SMT primitives. However, STIBP is not *always-on* in some CPUs, like Coffee Lake. Systems that do not support STIBP, such as Zen 1, have no option other than to disable SMT entirely, resulting in significant performance impact [2], [44].

Hardware-based isolation. Ultimately, the root cause of vBTI primitives is the lack of BPU state isolation between host and guest domains. Zen 5 CPUs introduce privilege domain tagging to distinguish between user and supervisor domains. A similar tagging to distinguish the host and the guest can prevent vBTI primitives and should be considered for future revisions of the Zen microarchitectures.

10. Related Work

There is a substantial body of research on microarchitectural security spanning decades, starting with timing side channels, in particular on caches [45], [46]. Acıiçmez et al. [47], [48] presented similar work but targeting early branch predictors. Evtyushkin et al. [31] later demonstrated that modern BTBs can be similarly exploited to break ASLR. Ge et al. [49] presented a classification of such (and many other) exploitable timing side channels through the shared resources and contexts.

Spectre. Kocher et al. [1] presented the first transient execution attack caused by branch misprediction. A host of other transient execution attacks then followed [50], [51], [52], [53], [54], [55], [56], [57]. Canella et al. [58] provide an evaluation and classification of many such attacks. We categorize prior speculative execution attacks into three groups based on their victim protection domain: (1) hypervisor attacks, (2) supervisor attacks, and (3) userspace attacks.

(1) Hypervisor attacks. The original work on Spectre [1], alongside later work [5], [8], provided proof of concept attacks against modified supervisor components of hypervisor software. Google researchers partially published an attack against KVM [59], based on Inception [6]. Our work, in contrast, presents the first real end-to-end attack where a malicious KVM guest targets *userspace* components of the hypervisor.

(2) Supervisor attacks. Supervisor mode has been an attractive target for BTI exploits early on [1]. While CPU vendors and operating system developers scrambled for mitigations, researchers continued to find gaps in the new defenses. Wikner and Razavi circumvented retpoline in Retbleed [2] and in-hardware mitigations in Phantom [43] and Inception [6] by invalidating assumptions about predictor training. BHI and follow-up work demonstrated repeatedly that same-mode training is possible despite various measures against it [3], [4], [5], breaking assumptions made by cross-privilege BTI mitigations.

(3) Userspace attacks. Various attacks against userspace have been proposed, in particular against browser sandboxing. Ragab et al. [60] generalized machine clears as a source of speculative execution and identified novel causes of machine clears that they exploited in the browser. Ret2spec [61] and Spectre Returns [21] both explore *Return Stack Buffer* (RSB) predictions to attack userspace victims in various scenarios. Spring [62] then showed that despite various system and in-browser mitigations, RSB predictions could still be exploited. More recent attacks also target other predictors to similarly leak secrets across sandbox boundaries [63], [64]. Wikner and Razavi [7] then were the first to demonstrate a full end-to-end cross-process attack. In contrast to previous attacks targeting userspace, we identified and exploited primitives that operate not only across context, but also across privilege and virtualization boundaries simultaneously.

11. Conclusion

We presented VMScape, the first practical Spectre-BTI attack from a malicious KVM guest on QEMU, leaking memory at $154\,\mathrm{B/s}$ on AMD Zen 5. Unlike previous Spectre-BTI attacks from a malicious guest, VMScape does not require any hypervisor code modification. VMScape achieves this using a novel vBTI primitive that we discovered by systematically exploring isolation gaps in the branch predictor state between the guest and host at different privilege levels on different AMD and Intel CPUs. Mitigating VMScape requires an IBPB after a `VMEXIT` when entering a userspace hypervisor, which introduces a marginal performance overhead.

Acknowledgments

We would like to thank the anonymous reviewers and our shepherd for their feedback. We would also like to thank the Intel PSIRT, AMD PSIRT and Linux security teams for coordination and collaboration on this issue. Furthermore, we thank Alexandra Sandulescu from Google for her feedback on our mitigation analysis. This work was supported by the Swiss State Secretariat for Education, Research and Innovation under contract number MB22.00057 (ERC-StG PROMISE).

References

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre Attacks: Exploiting Speculative Execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, May 2019, pp. 1–19.

[2] J. Wikner and K. Razavi, "RETBLEED: Arbitrary Speculative Code Execution with Return Instructions," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3825–3842.

[3] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, "Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 971–988.

[4] S. Wiebing, A. d. F. Tron, H. Bos, and C. Giuffrida, "InSpectre Gadget: Inspecting the Residual Attack Surface of Cross-privilege Spectre v2," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 577–594.

[5] S. Wiebing and C. Giuffrida, "Training Solo: On the Limitations of Domain Isolation Against Spectre-v2 Attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, May 2025, pp. 3599–3616.

[6] D. Trujillo, J. Wikner, and K. Razavi, "Inception: Exposing New Attack Surfaces with Training in Transient Execution," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 7303–7320.

[7] J. Wikner and K. Razavi, "Breaking the Barrier: Post-Barrier Spectre Attacks," in *S&P*, May 2025.

[8] S. Rüegge, J. Wikner, and K. Razavi, "Branch Privilege Injection: Compromising Spectre v2 Hardware Mitigations by Exploiting Branch Predictor Race Conditions," in *USENIX Security*, Aug. 2025, intel Bounty Reward, BlackHat USA presentation. [Online]. Available: Paper=https://comsec.ethz.ch/wp-content/files/bprc_sec25.pdfURL=https://comsec.ethz.ch/bprc

[9] M.-M. Bazm, M. Lacoste, M. Südholt, and J.-M. Menaud, "Isolation in cloud computing infrastructures: New security challenges," *Annals of Telecommunications*, vol. 74, no. 3, pp. 197–209, Apr. 2019.

[10] Advanced Micro Devices, Inc., "AMD64 APM, Volume 2: System Programming," Tech. Rep. 24593, Mar. 2024.

[11] "Intel Trust Domain Extension," Tech. Rep.

[12] "Kernel virtual machine," https://linux-kvm.org/page/Main_Page.

[13] "QEMU: A generic and open source machine emulator and virtualizer," https://www.qemu.org//.

[14] "Intel64 SDM, Volume 3 (3A, 3B, 3C & 3D): System Programming Guide," Tech. Rep.

[15] "Proxmox: Simplify your data center," https://proxmox.com/en/.

[16] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. Tullsen, "Pathfinder: High-Resolution Control-Flow Attacks Exploiting the Conditional Branch Predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. La Jolla CA USA: ACM, Apr. 2024, pp. 770–784.

[17] L. Li, H. Yavarzadeh, and D. Tullsen, "Indirector: High-Precision Branch Target Injection Attacks Exploiting the Indirect Branch Predictor," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2137–2154.

[18] Advanced Micro Devices, Inc., "Software Optimization Guide for the AMD Zen5 Microarchitecture," Tech. Rep. 58455, Aug. 2024.

[19] Y. Zhang and R. Sion, "Speculative Execution Attacks and Cloud Security," in *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*, ser. CCSW'19. New York, NY, USA: Association for Computing Machinery, Nov. 2019, p. 201.

[20] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 719–732.

[21] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. Abu-Ghazaleh, "Spectre Returns! Speculation Attacks using the Return Stack Buffer," in *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[22] "Indirect Branch Restricted Speculation," Tech. Rep.

[23] Advanced Micro Devices, Inc., "Amd64 Technology Indirect Branch Control Extension," Tech. Rep., Jul. 2018.

[24] "Single Thread Indirect Branch Predictors," Tech. Rep.

[25] "Providing secret data to QEMU," https://www.qemu.org/docs/master/system/secrets.html.

[26] H. Wang, M. Tang, Q. Wang, K. Xu, and Y. Zhang, "ZenLeak: Practical Last-Level Cache Side-Channel Attacks on AMD Zen Processors," 2025.

[27] Advanced Micro Devices, Inc., "AMD64 APM, Volume 3: General-Purpose and System Instructions," Tech. Rep. 24594, Mar. 2024.

[28] ——, "Software Optimization Guide for the AMD Zen4 Microarchitecture," Tech. Rep. 57647, Jan. 2023.

[29] "Speculative Return Stack Overflow (SRSO) — The Linux Kernel documentation," https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/srso.html.

[30] Advanced Micro Devices, Inc., "Technical Guidance for Mitigating Branch Type Confusion," Tech. Rep.

[31] D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, "Jump over ASLR: Attacking branch predictors to bypass ASLR," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2016, pp. 1–13.

[32] Documentation for /proc/sys/kernel. The Linux Kernel documentation. [Online]. Available: https://docs.kernel.org/admin-guide/sysctl/kernel.html#numa-balancing

[33] Configure NUMA-aware VMs — Google Distributed Cloud (software only) for bare metal. Google Cloud. [Online]. Available: https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/vm-runtime/numa

[34] How ESXi NUMA Scheduling Works. [Online]. Available: https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/vsphere-resource-management-8-0/using-numa-systems-with-esxi/how-esxi-numa-scheduling-works.html

[35] NUMA - Proxmox VE. [Online]. Available: https://pve.proxmox.com/wiki/NUMA

[36] "Branch History Injection and Intra-mode Branch Target Injection," https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/branch-history-injection.html.

[37] "UnixBench test suite," https://github.com/kdlucas/byte-unixbench.

[38] "Flexible I/O Tester," https://github.com/kdlucas/byte-unixbench.

[39] "Retpoline: A software construct for preventing branch-target-injection," https://support.google.com/faqs/answer/7625886.

[40] "VirtualBox: Powerful open source virtualization For personal and enterprise use," https://www.virtualbox.org/.

[41] "Firecracker: Secure and fast microVMs for serverless computing."

[42] "Crosvm: The ChromeOS Virtual Machine Monitor."

[43] J. Wikner, D. Trujillo, and K. Razavi, "Phantom: Exploiting Decoder-detectable Mispredictions," in *56th Annual IEEE/ACM International Symposium on Microarchitecture*. Toronto ON Canada: ACM, Oct. 2023, pp. 49–61.

[44] P. Michaud and A. Seznec, "A Comprehensive Study of Dynamic Global History Branch Prediction," Report, INRIA, 2001.

[45] W.-M. Hu, "Lattice scheduling and covert channels," in *Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, 1992, pp. 52–61.

[46] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *Advances in Cryptology — CRYPTO '96*, N. Koblitz, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 104–113.

[47] O. Acıiçmez, Ç. K. Koç, and J.-P. Seifert, "Predicting secret keys via branch prediction," in *Topics in Cryptology – CT-RSA 2007*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 225–242.

[48] O. Aciiçmez, c. K. Koç, and J.-P. Seifert, "On the power of simple branch prediction analysis," in *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*, ser. ASIACCS '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 312–320. [Online]. Available: https://doi.org/10.1145/1229285.1266999

[49] Q. Ge, Y. Yarom, D. Cock, and G. Heiser, "A survey of microarchitectural timing attacks and countermeasures on contemporary hardware," 2018.

[50] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading Kernel Memory from User Space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 973–990.

[51] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, p. 991.

[52] V. Kiriansky and C. Waldspurger, "Speculative buffer overflows: Attacks and defenses," 2018. [Online]. Available: https://arxiv.org/abs/1807.03757

[53] J. Stecklina and T. Prescher, "Lazyfp: Leaking fpu register state using microarchitectural side-channels," 2018. [Online]. Available: https://arxiv.org/abs/1806.07480

[54] D. Moghimi, "Downfall: Exploiting speculative data gathering," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 7179–7193. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/moghimi

[55] S. van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "RIDL: Rogue in-flight data load," in *S&P*, May 2019.

[56] C. Canella, D. Genkin, L. Giner, D. Gruss, M. Lipp, M. Minkin, D. Moghimi *et al.*, "Fallout: Leaking data on meltdown-resistant cpus," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2019.

[57] M. Schwarz, M. Lipp, D. Moghimi, J. Van Bulck, J. Stecklina, T. Prescher, and D. Gruss, "ZombieLoad: Cross-privilege-boundary data sampling," in *CCS*, 2019.

[58] C. Canella, J. V. Bulck, M. Schwarz, M. Lipp, B. von Berg, P. Ortner, F. Piessens, D. Evtyushkin, and D. Gruss, "A systematic evaluation of transient execution attacks and defenses," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 249–266. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/canella

[59] "Google CPU security research," https://github.com/google/security-research/tree/master/pocs/cpus.

[60] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, "Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1451–1468. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/ragab

[61] G. Maisuradze and C. Rossow, "ret2spec: Speculative execution using return stack buffers," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2109–2122. [Online]. Available: https://doi.org/10.1145/3243734.3243761

[62] J. Wikner, C. Giuffrida, H. Bos, and K. Razavi, "Spring: Spectre returning in the browser with speculative load queuing and deep stacks," in *WOOT*, 2022.

[63] J. Kim, D. Genkin, and Y. Yarom, "Slap: Data speculation attacks via load address prediction on apple silicon," in *S&P*, 2025.

[64] J. Kim, J. Chuang, D. Genkin, and Y. Yarom, "Flop: Breaking the apple m3 cpu via false load output predictions," in *USENIX Security*, 2025.

Appendix A.
Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

A.1. Summary

In this paper, the authors further investigate the susceptibility of x86 CPUs to Spectre-BTI vulnerabilities, focusing on virtualization boundaries. The authors found several problems in the isolation enforcement across such boundaries, leaving host to guest and guest to host attack surface. The paper analyzes this attack surface for different CPU classes and demonstrates attacks on Zen4 CPUs.

A.2. Scientific Contributions

- Independent Confirmation of Important Results with Limited Prior Research
- Identifies an Impactful Vulnerability
- Provides a Valuable Step Forward in an Established Field
- Other

A.3. Reasons for Acceptance

1) This paper is the first to analyze Spectre-BTI across virtualization boundaries. Presented results are impactful especially in cloud servers where virtualization is ubiquitous, and the proposed attacks are realistic. CPU vendors and OS developers have confirmed the existence of the vulnerabilities and are working on patches to address them.
2) The paper proposes new techniques for LLC evictions on Zen-class CPUs that can further aid the analysis of other side-channel attacks against such CPUs.
3) The paper presents an end-to-end attack on Zen4 CPUs that are widely-adopted in cloud machines.

A.4. Noteworthy Concerns

1) While the paper presents an analysis of different attacks across virtualization boundaries and with different CPUs, attacks are only demonstrated on the Guest to Host-User scenario and AMD Zen4 CPUs. This raises some doubts in reviewers about direct applicability to other scenarios.

VMSCAPE：在云环境中暴露并利用不完整的分支预测器隔离机制

Jean-Claude Graf† ETH Zurich
Sandro Rüegge† ETH Zurich
Ali Hajiabadi ETH Zurich
Kaveh Razavi ETH Zurich

†Equal contribution joint first authors

***摘要*——虚拟化是现代云基础设施的基石，能为客户提供所需的隔离保护。然而，这种隔离正受到推测执行攻击的威胁，CPU厂商试图通过将隔离范围扩展至分支预测器状态来缓解这一风险。我们的系统分析表明，这种扩展不幸并不彻底：虽然现有硬件防护措施已解决了宿主机中客户机控制分支预测这一最明显的案例，但我们发现了AMD Zen 1-5和Intel Coffee Lake处理器上若干新的Spectre *分支目标注入*（Spectre-BTI）攻击原语，其中某些攻击可使恶意客户机在用户空间执行时控制宿主机的间接分支预测。利用上述原语，我们开发了VMS CAPE——首个Spectre- BTI 攻击手段，能使恶意 KVM 客户机以154 B/s的速度从运行于AMD Zen 5服务器系统的未修改 QEMU 进程中泄漏任意内存，从而暴露磁盘加密与解密所需的密码密钥。我们对潜在缓解策略的分析表明：在常见场景下，通过选择性刷新分支预测器，可在不影响性能的前提下有效降低 VMS CAPE。**

1. 引言

Spectre v2或*分支目标注入*（BTI）[1]攻击近期备受关注，其典型攻击方式是从非特权用户处泄露特权内存[2]。3],[4], [5], [6], [7], [8]尽管这些攻击充分暴露了安全边界被突破的情况，但由于其假设攻击者仅能在用户系统上执行本地代码，因此在实际应用中的影响有限。更具威胁性的场景出现在云端：恶意的*虚拟机*（VM）可利用Spectre- BTI 漏洞泄露主机或其他虚拟机中的信息。然而，目前公开报道的此类攻击案例均要求攻击者在主机上控制代码运行[1]。5][8] 这是一个不切实际的假设。通过对虚拟化边界间分支预测状态隔离的系统性分析，我们发现了多种基于Spectre- BTI 的新攻击原语。随后，我们利用其中一种原语构建了首个Spectre-BTI 漏洞利用工具，该工具能够从运行默认配置下未修改软件的主机中泄露信息。

云中的Spectre- BTI 攻击。 以虚拟机形式存在的虚拟化技术，是云环境中安全隔离共置工作负载的主要机制，能够同时保护客户数据和底层基础设施[9]。Spectre攻击（尤其是Spectre- BTI）可通过滥用CPU内部共享的分支预测器状态来破坏这种隔离性[1]。在此场景下常见的威胁模型是恶意虚拟机对管理程序或其他虚拟机发起攻击。因此，硬件和软件供应商特别注重通过以下方式缓解此类威胁：对虚拟机与管理程序中学习到的分支预测条目进行差异化标记，并在切换不同虚拟机时清除分支预测器状态。而针对管理程序的混淆代理攻击所形成的残余攻击面，在不假设代码修改的情况下实际难以被利用[5]。本文研究的核心问题是：是否存在其他尚未被探索的、由恶意虚拟机发起的潜在攻击手段。

适用于云环境的新型 Spectre- BTI 攻击原语。 我们得出一个关键结论：现有的分支预测器隔离机制过于粗粒度——它们将虚拟机监控程序和虚拟机视为单一整体，而实际上它们各自拥有独立的权限级别。为探究现有分支预测器隔离机制如何处理不同虚拟化域中的这些权限级别，我们系统分析了 x86 CPU 中攻击者与受害者防护域的所有可能组合，即 *主机管理程序* (HS)、*主机用户* (HU)、*客户端管理程序* (GS) 和 *客户端用户* (GU)。我们的分析表明：尽管存在硬件层面的防护措施，所有 AMD Zen 处理器（Zen 1 至 Zen 5）和 Intel Coffee Lake 架构均易受新型 *基于虚拟化的 Spectre- BTI*（vBTI）攻击原语的影响，该攻击发生在客户端用户与主机用户之间，我们将其称为 vBTI $_{GU\rightarrow HU}$ 和 vBTI $_{HU\rightarrow GU}$。vBTI $_{GU\rightarrow HU}$ 使得恶意虚拟机能够对主机上运行的用户进程发起新的 Spectre- BTI 攻击；而 vBTI $_{HU\rightarrow GU}$ 则在假设主机具有恶意性且客户机部署了硬件辅助隔离机制（例如 *安全加密虚拟化 - 安全嵌套分页* (SEV -SNP) [10] 或 *信任域扩展* (TDX) [11]）的情况下，同样能引发新的 Spectre- BTI 攻击。此外，我们还发现了其他需要针对本文将讨论的其他攻击场景加以考虑的 vBTI 原语。

vmscape。 为展示我们vBTI原语的实际可行性和严重性，我们提出了 VMS CAPE——首个基于Spectre的端到端攻击方案：恶意客户机用户无需任何代码修改且在默认配置下，即可从主机域的虚拟机监控程序中泄露任意敏感信息。我们的攻击目标是广泛使用的*内核虚拟机*（KVM）/ QEMU [12][13]作为虚拟机监控程序，特别是 QEMU 作为主机中虚拟机监控程序的用户空间组件。虽然vBTI $_{GU\rightarrow HU}$原语验证了该攻击方案的可行性，但我们还解决了若干额外挑战，从而能够对AMD Zen 4和Zen 5 CPU实施可靠的端到端攻击。例如，实现任意内存泄漏需要足够大的推测窗口；然而，对于客户机用户而言，获取大型推测窗口极具挑战性——由于无法访问物理内存或对 QEMU 进行代码修改，他们无法直接清除受害分支指针。为解决这一问题，我们对AMD Zen 4和Zen 5的缓存层次结构进行了逆向工程，为其非全包含的*最后一级缓存*（LLC）构建了驱逐集，从而为 VMS CAPE提供了足够大的推测窗口，使其能够成功应用我们在 QEMU 中发现的技术手段。

VMS CAPE在AMD Zen 5处理器上可以每秒154字节/的速度泄露 QEMU 进程的内存。我们使用VM S CAPE在102秒内定位并泄露了机密数据，例如提取用于磁盘加密/解密的密码学密钥。分析表明，缓解 VMS CAPE需要在特定条件下显式清空*分支预测单元*（BPU）的状态；具体而言，在进入用户空间虚拟化层之前，每个 VM-EXIT 都必须设置*间接分支预测屏障*（IBPB）。评估结果显示，Linux内核维护者为应对 VMS CAPE而开发的此类缓解措施，在常见场景下仅会产生轻微的性能开销。

贡献。 我们做出以下贡献：

- 这是首次对所有可能的虚拟化与权限边界下的分支预测器隔离现象进行系统性分析，从而发现了即使在近期CPU防护措施下仍能有效运作的新基本单元。
- 首个针对未修改软件的从客户端到主机的Spectre- BTI 攻击——名为 VMS CAPE——利用我们新发现的vBTI $_{GU\rightarrow HU}$原语，以及关于AMD Zen 4和Zen 5处理器缓存驱逐机制的新见解，从而扩大了推测窗口。
- 针对多种微架构的缓解方案进行分析，最终提出 IBPB -on- VMEXIT 作为减轻 VMS CAPE影响的基础方法。

责任性披露。 我们已于2025年6月7日向AMD和Intel PSIRT 披露了 VMS CAPE漏洞，该漏洞的披露信息在2025年9月11日前仍处于保密状态。Linux 维护团队根据我们的 IBPB -on-VMEXIT 建议，通过补丁修复了 VMS CAPE漏洞。我们已验证这些补丁能有效阻止vBTI原语的攻击行为。 VMS CAPE漏洞的追踪编号为 CVE -2025-40300。更多相关信息（包括实验及漏洞利用代码的源码）可访问https://comsec.ethz.ch/vmscape获取。

2. 背景

我们介绍了Linux中基于*内核虚拟机*（KVM）的虚拟化堆栈背景（第2.1节）、分支预测机制（第2.2节）、Spectre- BTI 攻击（第2.3节）及其相应的缓解措施（第2.4节）。最后，我们强调了在云环境中研究 Spectre- BTI 攻击的必要性（第2.5节）。

2.1. KVM 虚拟化

硬件虚拟化扩展技术，例如AMD SVM [10]和Intel VT-x[14]，能够为在同一处理器上同时运行多个操作系统提供硬件支持，以极低的性能开销实现强大的隔离保障。*虚拟机监控程序*为每个*客户虚拟机*营造出完全掌控物理系统的错觉。在Linux系统中， KVM 充当虚拟机监控程序，并作为内核模块运行于*主机*之上；该程序依赖用户空间组件来管理虚拟机的生命周期及模拟设备，我们将此用户空间进程称为*虚拟机监控程序的用户模式组件*（Hypervisor $_{user}$）。 QEMU [13]是Linux系统中广泛使用的虚拟机监控程序$_{user}$[15]，其功能包括创建、启动、停止和迁移虚拟机，并模拟所需的硬件设备。每个客户机*虚拟CPU*（vCPU）在虚拟机监控程序$_{用户空间}$中对应一个专用线程，这使得主机内核能够像调度普通主机进程一样来安排客户机进程的执行——因为在Linux中，调度器对线程和进程的处理方式是相似的。因此，客户机线程的执行会与其他客户机线程及主机进程的执行交替进行。

进入与退出虚拟机。 vCPU线程以“客户机模式”或“主机模式”运行。要从主机模式切换到客户机模式，该线程会向 KVM 内核模块发出KVM_RUN *ioctl*指令，并传递其vCPU的指针。根据平台不同， KVM 会执行vmlaunch/vmresume（在Intel平台上）或 VMRUN（在AMD平台上）指令。CPU从vCPU中恢复自身状态并启动客户机执行。当客户机执行敏感或特权指令时，CPU会拦截该指令并触发 VMEXIT 。CPU将内部状态保存回vCPU结构后，在 KVM 中继续执行。根据 VMEXIT 类型的不同， KVM 要么直接处理退出操作，要么通过KVM_EXIT将其委托给虚拟机监控程序$_{用户}$。

保护域。 为确保机密性和完整性，计算系统必须强制实施在不同保护域中运行的实体之间的隔离。在许多场景下，重点在于区分*用户*与*监督者*域（即环0与环3）。然而，在考虑虚拟化环境时，这一抽象概念必须扩展以涵盖客户机执行。如图1所示，用户域包含*主机用户*（HU）和*客户机用户*（GU）域，而监督者域则涵盖*主机监督者*（HS）和*客户机监督者*（GS）域。

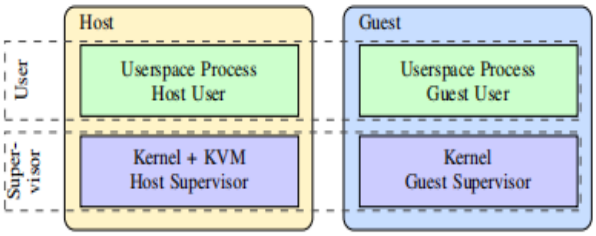


图1。在配备虚拟机（VM）的x86-64系统中，保护域包括HU、HS、GU和GS，其范围超越了用户模式和管理员模式。

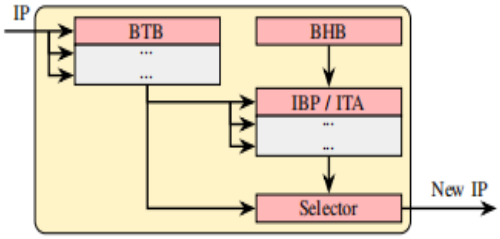


图2。参与预测间接分支的 BPU 组件。 BTB 提供静态预测，而 IBP /ITA 则将目标与存储在Intel/AMD处理器 BHB 中的路径历史信息相关联。

2.2. 分支预测

CPU优化的核心之一在于对分支指令及其目标地址的精准预测。现代CPU中的*分支预测单元*（BPU）结构极为复杂，由多种针对不同分支类型和上下文设计的预测器组成。虽然*静态预测器*仅考虑分支的源地址，但*动态预测器*会利用分支路径的历史信息作为额外上下文参考[2]。3], [6], [7],[16], [17] *间接分支* 会将控制流转移至由寄存器或内存指定的目标地址。这使得分支指令能够指向多个目标地址， BPU 会尝试根据分支路径的历史记录进行关联判断。英特尔CPU采用 *分支目标缓冲区*（BTB）进行静态预测，以及 *间接分支预测器*（IBP）进行动态预测。在可用情况下，动态预测优先于静态预测 [8]。17]在AMD CPU上，动态预测器被称为*间接目标数组*（ITA），当分支操作涉及多个目标地址时使用该机制；否则仅使用 BTB [18]。路径历史记录会同时存储在Intel和AMD架构的*分支历史缓冲区*（BHB）中。图2展示了间接分支预测所涉及的结构。

2.3. Spectre-BTI

Spectre Branch Target Injection 或 Spectre- BTI [19] 是一类推测执行攻击，攻击者可通过利用间接分支预测错误，在保护域边界之间泄露数据。攻击者使 BPU 将*目标分支*（作为*推测装置*的一部分）预测为*披露装置*，从而引发*瞬态执行*。在披露装置中，秘密信息的访问方式使得攻击者能够通过*侧信道*（通常基于缓存的 F LUSH+R ELOAD [20]）事后推断出该秘密。为导致目标分支的预测错误，攻击者需使用一个与目标预测器中的目标分支*发生冲突*的*训练分支*来*训练* BPU 。若要针对特定预测器，攻击者还需模拟训练分支中目标分支的路径历史。

此前关于Spectre- BTI 的大多数研究都集中在以下场景：非特权主机用户进程针对主机监督程序发起攻击[1]。2], [3], [5], [6], [21]相比之下，用户进程之间的攻击却鲜少受到关注[7]。

2.4. Spectre- BTI 缓解措施

Spectre-*分支目标注入*（BTI）攻击要求攻击者控制用于预测受害者的 BTB 状态，这意味着攻击者与受害者域之间需要共享 BPU 状态。*主域隔离*技术旨在通过阻止关键保护域边界之间的 BPU 状态共享来缓解 BTI 攻击。*间接分支限制推测*（IBRS）[22]可防止低权限域（例如用户）的分支影响高权限域（例如管理员）的分支，从而保护内核免受用户模式攻击者的侵害。英特尔和AMD已部署*增强型 IBRS*（eIBRS）[22]和*自动 IBRS*（AutoIBRS）[23]以改进 IBRS ；这两种防护措施均为*持续激活*，且无需在权限转换时进行昂贵的模型特定寄存器写入操作。这些对 BPU 进行*清洗*的防护措施可在潜在恶意条目影响其他保护域中的受害者之前将其清除。*间接分支预测屏障*（IBPB）允许开发人员在那些原本无法充分隔离的域转换期间显式清除 BPU 状态。在启用了*Si多线程支持*（SMT）的CPU上， BPU 状态也可能在*同级线程*之间共享。*单线程间接分支预测器*（STIBP）[24]是一种缓解措施，可限制 SMT 线程之间的 BPU 状态共享。

不同微架构提供的缓解措施各不相同，默认的缓解方案取决于硬件支持和内核配置。表1列出了我们在评估的系统中可用的缓解措施，并标明了这些措施在Ubuntu 24.04系统上是否默认启用。

2.5. 动机

虽然用户对内核发起的Spectre- BTI 攻击属于安全漏洞，但此类攻击需要在受害机器上执行代码，从而限制了其影响范围。可以说，Spectre- BTI 更具威胁性的应用场景在于云端环境——攻击者可轻松租用虚拟机，并针对虚拟机监控程序或其他虚拟机实施攻击。相关前期研究[1]5]研究人员已探讨了恶意攻击者针对虚拟机监控程序发起攻击的情形。然而，此类攻击需通过修改虚拟机监控程序的代码来植入恶意组件，这极大限制了其实际应用价值与攻击效果。由此引出一个关键问题：*是否能在不做出过于严苛假设的前提下，跨虚拟化边界实施Spectre- BTI 攻击？*

TABLE 1. EVALUATED MICROARCHITECTURES AND THEIR ISOLATION MECHANISMS AGAINST SPECTRE-BTI.

Vendor	CPU	Year	Codename	Microarchitecture	Microcode	Isolation Mechanism			
						IBPB	IBRS	eIBRS/AutoIBRS	STIBP
Intel	Core i7-8700	2017	Coffee Lake S	Coffee Lake	0xfa	◓	●	○	◓
	Core i7-13700K	2022	Raptor Lake	Raptor Cove	0x12c	◓	○	●	●
				Gracemont	0x12c	◓	○	●	●
AMD	Ryzen 5 1600X	2017	Summit Ridge	Zen 1	0x8001137	◓	●	○	○
	EPYC 7252	2019	Rome	Zen 2	0x8301038	◓	●	○	●
	EPYC 7413	2021	Milan	Zen 3	0xa0011d3	◓	●	○	●
	Ryzen 7 7700X	2022	Raphael	Zen 4	0xa601209	◓	◓	●	●
	Ryzen 5 9600X	2024	Granite Ridge	Zen 5	0xb404023	◓	◓	●	●

○ Not available ◓ Available ● Available and Default (Ubuntu 24.04)

3. 威胁模型

攻击者的目标是推断跨越主机-客户机虚拟化边界的信息。具体而言，我们考虑以下三种威胁模型：首先，$TM_{G\rightarrow H}$描述了控制客户机虚拟机的攻击者针对主机系统的场景；其次，$TM_{H\rightarrow G}$表示恶意主机针对系统上运行的客户机发起攻击——这种威胁模型在涉及依赖英特尔*信任域扩展*（TDX）或AMD *安全加密虚拟化-安全嵌套分页*（SEV - SNP）来抵御恶意主机的客户机时尤为相关；第三，恶意客户机并非直接攻击主机，而是可能攻击另一个客户机，这体现在$TM_{G1\rightarrow G2}$模型中。我们假设采用了硬件虚拟化扩展技术：AMD使用 SVM [10]，Intel使用VT-x[14]。此外，我们假设所有组件（特别是主机和客户机内核以及虚拟机监控程序$_{用户端}$）均不存在软件漏洞。最后，我们假设这些系统采用了针对Spectre- BTI 的所有默认安全防护措施，具体如表1所示。

4. 挑战概述

现有的Spectre- BTI 缓解措施旨在跨保护域边界隔离 BPU 状态。尽管过去已研究过恶意客户机攻击虚拟机监控程序这一最典型的案例[1]，5]目前仍缺乏针对虚拟化环境中 BPU 状态隔离的系统性分析，该分析需综合考虑客户机与主机的不同权限级别。这引出了我们的首要挑战：

挑战C1： 识别 BPU 在虚拟化环境中保护域隔离机制存在的漏洞。

为应对这一挑战，我们需要仔细评估 BPU 中各个结构的隔离效果。在第5节中，我们系统性地测试了 BTB 和 IBP /ITA在考虑客户机与主机不同权限级别时所提供的隔离能力。系统分析表明，许多近期提出的微架构易受某些*基于虚拟化的Spectre- BTI（vBTI）原语*攻击影响，这说明 BPU 缺乏有效的主机-客户机隔离机制。

然而，利用这些新型vBTI基本单元所带来的具体后果及潜在攻击风险尚不明确。我们的下一个挑战是确定在何种特定情境下，攻击者能够在不同的威胁模型中实施有效的攻击。

挑战C2： 在虚拟化环境中，理解并识别受多孔 BTB 隔离机制影响的不同威胁模型。

应对这一挑战需要对攻击者利用已识别的vBTI原语所需条件进行详细分析。在第6节中，我们基于第3节提出的三种威胁模型，提出了五种新型攻击场景。这些攻击场景使攻击者能够利用所识别的原语跨越虚拟化边界对受害者发起攻击。其中，我们重点研究了一种新型跨权限vBTI原语的攻击方式：恶意客户机攻击虚拟机监控程序$_{用户}$以泄露敏感信息（例如云基础设施密钥）。尽管底层推测原语在概念上较为简单，但构建实用的端到端攻击方案仍面临挑战。

挑战C3： 实现跨虚拟化边界的实际应用与数据泄露。

客户机与主机软件层之间的交互有限，这使得寻找合适的推测工具和可靠的侧信道变得极具挑战性。实现有效数据泄露的关键在于确保足够的推测窗口宽度。然而，在现代AMD Zen微架构（尤其是Zen 4和Zen 5）上实现有效的缓存驱逐仍是一个未解决的问题。第7节详细阐述了在AMD Zen 4和Zen 5平台上构建首个可靠缓存驱逐机制的方法。基于这些研究成果，第8节开发了VMS CAPE——一种实用的Spectre漏洞利用方案，它允许客户机通过我们新发现的vBTI原语之一，在虚拟化边界之外泄露主机内存。

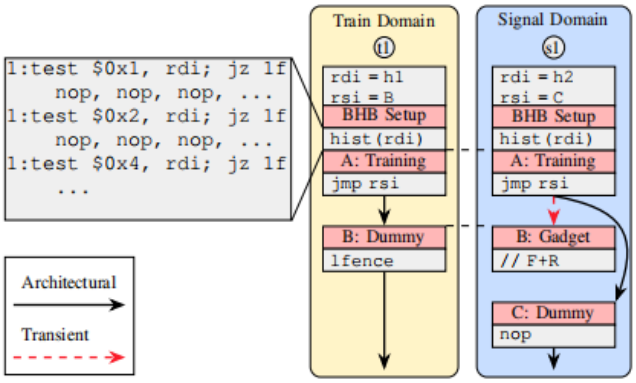


图3. vBTI实验中的训练（t1）与信号传输（s1）轨迹使我们能够评估BPU 在训练域与信号传输域边界之间的状态隔离能力。间接分支的源节点*A*和目标节点*B*在两个域中映射至相同的虚拟地址。每个间接分支前均会执行 BHB 设置——即通过rdi中的种子值生成一系列分支操作，使两条轨迹中的 BHB 状态保持一致。这确保了 BPU 无法区分训练分支与信号传输分支，因为两者会在 BTB 及ITA/ IBP 阶段发生冲突。若隔离机制失效，信号传输分支将偏离由rsi定义的架构目标，并在*B*处临时执行已训练的目标指令。我们通过一种信息披露机制来检测这种误判现象。

5. 虚拟化环境下的 BPU 隔离

我们分析了在虚拟化环境中，针对多种微架构跨越保护域边界进行间接分支预测的隔离性能。为评估整体隔离保障能力，我们需要分别考察静态与动态间接分支预测器的隔离效果，并测试x86架构硬件虚拟化环境下四个基本保护域（即HU、HS、GU和GS）之间的相互干扰。所有评估系统详见表1。近期英特尔Raptor Lake等处理器采用异构设计，包含两种微架构：一种专为高性能任务优化，另一种则适用于低功耗场景。

5.1. 方法论

为评估间接分支的跨域 BPU 干扰，我们采用经典的Spectre- BTI [1]实验方法（如图3所示）。我们在一个域进行训练，并在另一个域使用F LUSH+R ELOAD侧信道[20]生成干扰信号。

本次分析要求我们在所有四个评估域中执行任意代码。对于主机系统（HS）的代码执行，我们实现了一个内核模块，允许用户以超级管理员身份运行任意用户代码；这需要在主机系统上禁用*超级管理员模式执行防护*（SMEP）和*超级管理员模式访问防护*（SMAP）。对于客户机系统（guest）的执行，则依赖于Linux内核内置的KVM 自测试基础设施。这一简化的虚拟机监控程序使我们能够将代码映射到任意客户机虚拟地址并执行。然而，KVM 自测试默认以GS权限运行，因此我们需要扩展该框架，并实现额外的权限转换机制以达到GU权限。

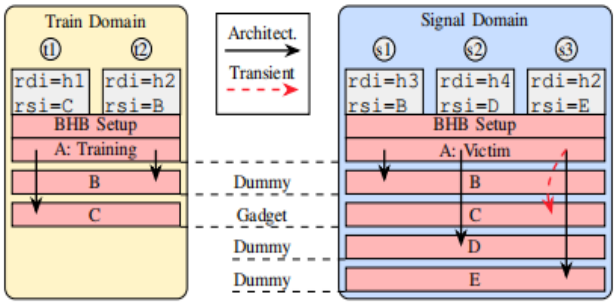


图4。通过组合不同的训练轨迹（t1、t2、s1、s2），我们可以诱导BPU 使用来自 BTB 或 IBP /ITA的信号轨迹s3的预测结果。分支源*A*与目标*B*和*C*的虚拟地址在两个域中完全一致。

在我们的评估过程中，至关重要的是要验证所有参与间接分支预测的结构是否处于隔离状态。这些结构包括英特尔架构中的 BTB 和 IBP，以及AMD架构中的BTB 和ITA。逆向工程要求我们控制 BHB 的状态，以引发或防止 IBP /ITA中的冲突。如图3所示，我们通过执行基于控制种子生成的伪随机分支序列来初始化BHB 。这段历史记录设置代码直接置于训练分支和受害分支之前，从而能够精确控制这些分支处的 BHB 状态。

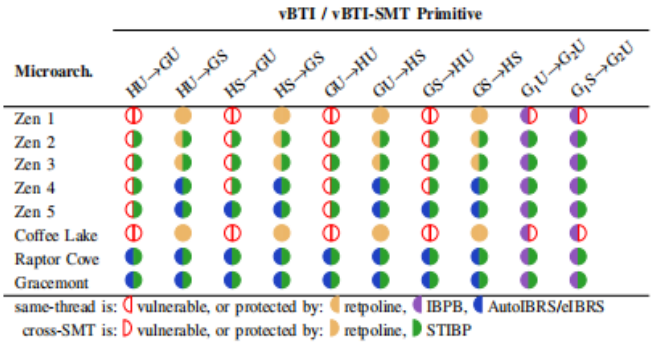
请注意，本节所述的所有修改均仅用于逆向工程目的。第8节中介绍的最终攻击无需任何软件修改，并可在表1所列的默认防护措施下执行。

BTB 预测。 如图2所示，多个结构参与间接分支的预测。因此，需要精确的实验设置以确保预测结果来自BTB 而非 IBP /ITA。英特尔始终会在 BTB 中检查命中情况，但在可用时优先考虑 IBP 的命中结果[17]；而AMD则使用 BTB 来预测所有仅接触过单一目标的间接分支，否则会优先采用ITA的预测结果[18]。

在训练域中，从*A*到*B*的训练分支执行时，会在Intel和AMD架构上均向 BTB 插入一条记录（图3 t1）。切换至信令域后，我们在*A*到*C*处执行信令分支（s1）。由于*A*在训练域与信令域中是相同的虚拟地址，BTB 无法区分两者。此外，使用种子值h_1可实现不同的 BHB 状态。$\neq h_2$ 确保英特尔处理器上的 IBP 无法提供预测结果，而自动切换至 BTB 进行处理；而在AMD处理器上，由于分支源*A*迄今仅遇到目标*C*，因此ITA中尚未创建任何条目，导致该分支同样由 BTB 进行预测。为确保状态清晰，我们在每次迭代之间插入一个 IBPB 。本实验中检测到的信号表明该机制存在vBTI攻击风险。

IBP /ITA预测。测试动态预测器的关键挑战在于：必须确保预测结果并非源自 BTB 。为此，我们需要执行特定的验证流程。

TABLE 2. SUMMARY OF THE OBSERVED VBTI PRIMITIVES FOR ALL EVALUATED SYSTEMS. EMPTY HALVE CIRCLES INDICATE VULNERABLE, WHILE COLORED ONES INDICATE THE DEFAULT MITIGATION PREVENTING A LEAK.



训练轨迹如图4所示。在训练域中，我们在A处执行训练分支两次，每次针对不同的目标。为迫使 BPU 区分这两次调用，我们设置了不同的历史种子$h_1 \neq h_2$。因此，不仅会在 BTB 中创建条目，还会在 IBP /ITA内部创建条目。

从训练域切换到信号域可能会导致 BTB 条目失效或被隔离。然而，在Intel和AMD架构中，都需要 BTB 中的有效条目才能让 IBP /ITA进行预测。因此，我们必须确保在切换至信号域后仍存在相应的 BTB 条目。在Intel架构中，仅需在A（s1）处遇到一次间接分支即可使后续分支（s3）由 IBP 进行预测；而在AMD架构中，则需要先出现指向不同目标的额外分支（s2），才能由ITA对最终分支（s3）进行预测。如果本实验中在当前域转换处检测到信号，但在先前域未检测到信号，则表明 BTB 与 IBP /ITA之间存在不平衡的隔离现象。

5.2. 原始评估

为评估隔离保证性，我们系统性地测试了虚拟化与权限边界之间所有保护域的组合。我们启用了表1中列出的所有默认缓解机制。此外，我们将所有实验分别以间接跳转和间接调用作为训练指令与信号指令各重复一次，从而评估两者是否被相似预测且均能实现有效隔离。结果表明间接跳转与间接调用之间无差异，说明二者均受到同等程度的缓解措施处理；因此在本文后续内容中我们不再区分二者。同时观察到干扰信号在客户机与主机之间呈对称分布，表明隔离机制在双向传输中具有等效性。我们采用符号vBTI $_{X\rightarrow Y}$表示一种训练域为X、信号域为Y的vBTI原语。

表2展示了各微架构对vBTI原语的敏感性。由于 BTB 与 IBP /ITA在漏洞易感性方面未发现差异，详见表2。

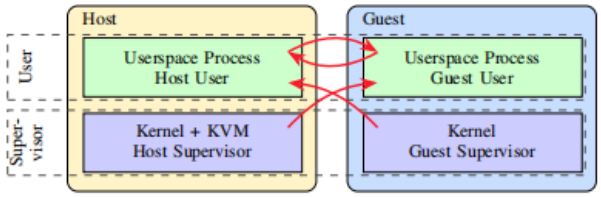


Figure 5. Effective protection domain isolation on Zen 4, susceptible to vBTI$_{GU\rightarrow HU}$, vBTI$_{GS\rightarrow HU}$, vBTI$_{HU\rightarrow GU}$, and vBTI$_{HS\rightarrow GU}$.

无需区分它们。我们观察到，除 Intel Raptor Cove 和 Gracemont 外，所有微架构都易受某些 vBTI 原语攻击。最值得注意的是，Zen 1 至 Zen 5 都易受 vBTI $_{HU\rightarrow GU}$ 和 vBTI $_{GU\rightarrow HU}$ 攻击；而 Zen 1 至 Zen 4 还额外易受 vBTI $_{HS\rightarrow GU}$ 和 vBTI $_{GS\rightarrow HU}$ 攻击（如图 5 所示）。虽然 Raptor Cove 和 Gracemont 已得到充分保护，但 Coffee Lake 对于 Zen 1 至 Zen 4 的上述原语同样存在漏洞。然而，我们发现没有任何微架构会受到针对 HS 和 GS 域的任何原语攻击。

观察结果 O1。 尽管已启用最新的缓解措施，所有 AMD Zen 微架构和 Intel Coffee Lake 都易受 vBTI $_{GU\rightarrow HU}$ 与 vBTI $_{HU\rightarrow GU}$ 的影响。此外，Zen 1 至 Zen 4 以及 Coffee Lake 均受到 vBTI $_{GS\rightarrow HU}$ 和 vBTI $_{HS\rightarrow GU}$ 的影响。

BPU 隔离机制。 AMD Zen 4和Zen 5采用AutoIBRS作为 Spectre- BTI 缓解措施。在监督模式下，该机制会阻止对间接分支目标的推测执行，直至这些目标得到解析[5]。6]通过在禁用AutoIBRS的情况下重新运行之前的实验，我们可以更深入地了解Zen 4和Zen 5微架构上的隔离机制实施情况。在缺乏AutoIBRS的情况下，Zen 4会受到所有vBTI原语的影响；而Zen 5则不受跨越用户与监督者边界原语的影响。这表明AMD Zen 5 CPU在AutoIBRS基础上还具备一种额外的原生隔离机制，其功能相当于为预测条目设置单比特权限级别标签。

观察结果 O2. 在 Zen 5 架构中，分支预测条目均标注了单比特权限级别，以区分用户域和监督者域。

然而，单个比特位不足以区分虚拟化环境中存在的四个域。因此，仍需使用AutoIBRS来分离特定域（例如HS域和GS域）。

观察结果O3。 尽管 BTB 中增加了隔离位，Zen5上的 BPU 仍无法区分主机域与客户机域，因此仍依赖 AutoIBRS来保护监督域。

SMT 隔离。 为补充先前测试同线程隔离的实验，我们采用改进后的实验来验证vBTI- SMT 原语。我们的训练过程会迭代执行训练分支（图3 t1）；随后信号处理过程绑定至SMT1兄弟核心，并使用不同历史数据执行信号分支（t2）。信号处理完成后，我们通过 IBPB刷新 BPU，使其*忽略*设备目标信息。如表2所示：虽然Zen 2至Zen 5及Raptor Lake架构上的 STIBP 能成功隔离兄弟线程，但Zen 1架构缺乏 STIBP 功能，导致vBTI- SMT $_{HU\rightarrow GU}$、vBTI- SMT $_{GU\rightarrow HU}$、vBTI- SMT $_{HS\rightarrow GU}$及vBTI- SMT $_{GS\rightarrow HU}$等原语无法实现隔离。此外我们发现，尽管Coffee Lake支持 STIBP，但默认并非始终启用该功能，从而允许相同的跨 SMT 原语被调用。

观察结果 O4。 在 Zen 1 上 STIBP 完全缺失，而在 Coffee Lake 上仅条件性启用；这两种微架构均对 vBTISMT $_{GU\rightarrow HU}$、vBTI- SMT $_{HU\rightarrow GU}$、vBTI- SMT $_{GS\rightarrow HU}$ 及 vBTI- SMT $_{HS\rightarrow GU}$ 存在漏洞。

6. 不同威胁模型下的剥削行为

我们的实验表明，某些虚拟化边界未能得到充分隔离，导致分支目标在这些边界之间发生干扰。在本节中，我们将研究结果与第3节提出的三种威胁模型相结合，并讨论了五种现实且此前未被探索的场景——这些场景中上述发现会导致具体的安全漏洞。我们用$A \rightarrow B \mid V$表示每种攻击场景，其中A和B分别代表攻击者和受害者的防护域，而V指代具体目标（例如主机进程）。

6.1. 从访客到主机的攻击

我们分析的第一个威胁模型是 TM $_{G\rightarrow H}$，该模型描述了恶意访客针对主机系统的攻击行为。在此模型中，我们识别出三种具体场景。

S1 G → H | 主机进程。 在此场景中，恶意来宾进程会针对主机用户进程泄露机密信息（例如 SUID 进程的密码哈希值）[7]。受害进程可能在来宾进程被抢占后被调度到同一硬件线程上（即*同一线程*），也可能被调度到同一物理核心上的 SMT 兄弟线程上（即*跨 SMT 线程*）。在同一线程情况下，除了缺乏来宾到主机的 BTB 隔离外，该攻击还要求任务切换期间不存在 BTB 清理机制。根据我们的逆向工程观察，我们发现Zen 1至Zen 5以及Intel Coffee Lake微架构缺乏足够的隔离性，易受vBTI $_{GS\rightarrow HU}$和vBTI $_{GU\rightarrow HU}$原语的影响。对于跨 SMT 攻击，来宾到主机的 BTB 干扰必须与兄弟线程间的共享预测器状态相结合，这由vBTI-SMT $_{GS\rightarrow HU}$和vBTISMT $_{GU\rightarrow HU}$原语所体现。这一条件在Zen 1和Coffee Lake微架构上均得到满足。

尽管这些攻击在理论上是可行的，但实际实施时面临诸多挑战。基于客户机的攻击者通常对主机用户进程的控制力有限，因此难以可靠地操控或与其交互。在客户机与任意主机用户进程之间建立可靠的侧信道尤为困难，因为攻击者既缺乏空间控制也缺乏时间控制。具体而言，攻击者无法决定目标进程何时何地被调度执行，这进一步增加了精确利用攻击漏洞的难度。

S2 G → H | Hypervisor $_{user}$。 该场景考虑了一个恶意客户机针对其自身的 Hypervisor $_{user}$。这要求 VMEXITs 和 KVM_EXITs 的 BTB 清理不足，同时在同一线程场景下客户机与主机之间缺乏 BTB 状态隔离。与一般的 G → H | Host Process 攻击相比，此处的攻击方式更为简单：客户机可故意触发由其 Hypervisor $_{user}$ 处理的 VMEXITs，从而在相同 SMT 场景下实现频繁的攻击者-受害者交互；跨 SMT 场景也更具实际可行性——通常调度机制通过降低攻击者与受害者共处的概率来缓解 crossSMT BTI 攻击，但在此情况下，受害者会执行同一客户机的 Hypervisor $_{user}$，使得共处概率显著增加。尽管攻击者仅能访问映射到虚拟机监控程序$_{用户}$地址空间的内存，但其中仍可能包含敏感数据，例如用于网络或存储的云基础设施密钥[25]。我们发现Zen 1至Zen 5及Coffee Lake架构均存在此类漏洞，其中Zen 1和Coffee Lake还存在跨 SMT 攻击风险。

观察结果O5： Cross- SMT G → H | Hypervisor $_{用户}$ 攻击更容易发生，因为攻击者与受害者属于同一客户机执行环境，这使得共置攻击的可能性显著增加。

S3 G → H | 客户端内核。 在此场景中，攻击者利用虚拟机监控程序$_{用户}$作为受误导的代理，从自身客户实例中提取内核内存。因此，该场景假设攻击者是运行在客户机上的无特权进程。这是 G → H | 虚拟机监控程序$_{用户}$ 场景的一种变体，但其泄露对象并非虚拟机监控程序$_{用户}$ 的密钥，而是映射在虚拟机监控程序$_{用户}$ 中的客户机自身内存。除要求虚拟机监控程序$_{用户}$ 对客户机内存进行映射外，其余前提条件与 G → H | 虚拟机监控程序$_{用户}$ 场景完全相同。具体而言，这些前提条件包括缺乏客户机与主机之间的 BTB 隔离机制，以及对 VMEXIT 和 KVM_EXIT 事件的清理处理不充分。

观察结果 O6： 近期推出的多种微架构（包括 Zen 5）均易受 G → H| Guest Kernel 攻击的影响。

TABLE 3. ATTACK SCENARIOS BY THREAT MODELS, INDICATING REQUIREMENTS, TARGET, AND VULNERABLE MICROARCHITECTURES.

Scenario	Target	SMT	Required Primitives	Exploitable on: Zen 1	Zen 2	Zen 3	Zen 4	Zen 5	Coffee Lake	Raptor Cove	Gracemont
(S1) G→H | Host Process	Host user process	Same	$vBTI_{GS\to HU}$ or $vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GS\to HU}$ or $vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S2) G→H | $Hypervisor_{user}$	$Hypervisor_{user}$	Same	$vBTI_{GS\to HU}$ or $vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GS\to HU}$ or $vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S3) G→H | Guest Kernel	Guest supervisor	Same	$vBTI_{GU\to HU}$	✓	✓	✓	✓	✓	✓	✗	✗
		Cross	$vBTI\text{-}SMT_{GU\to HU}$	✓	✗	✗	✗	✗	✓	✗	✗
(S4) H→G | SEV-SNP Process	Guest user process	Same	$vBTI_{HU\to GU}$ or $vBTI_{HS\to GU}$	-	-	✓	?	✗	-	✗	✗
		Cross	$vBTI\text{-}SMT_{HU\to GU}$ or $vBTI\text{-}SMT_{HS\to GU}$	-	-	✗	✗	✗	-	✗	✗
(S5) $G_1\to G_2$ | Guest Process	Other guest user process	Same	$vBTI_{G_1U\to G_2U}$	✗	✗	✗	✗	✗	✗	✗	✗
		Cross	$vBTI\text{-}SMT_{G_1U\to G_2U}$	✓	✗	✗	✗	✗	✓	✗	✗

✓ Vulnerable ✗ Not vulnerable - Not applicable ? Not supported on the tested system

6.2. 从主机到客户端的攻击

我们分析的第二个威胁模型是 $TM_{H\to G}$，该模型中恶意主机会针对客户端发起攻击。

(S4) H → G | SEV -SNP 过程。 在此场景中，恶意主机会针对客户机发起攻击——该客户机通常伪装成敌对主机，并采用基于硬件的隔离技术（如 SEV -SNP [10] 或 TDX [11]）。与 $TM_{G\to H}$ 不同，攻击者能够控制客户线程的调度，从而对受害者拥有更多控制权。由于这些技术会引入额外的安全措施，在未部署 SEV -SNP/ TDX 的系统上，仅凭工作中的 $vBTI_{HS\to GU}$、$vBTI_{HU\to GU}$、$vBTI\text{-}SMT_{HS\to GU}$ 及 $vBTISMT_{HU\to GU}$ 原语不足以判定不同微架构的漏洞存在性。然而我们已确认：在 Zen 3 架构上，默认情况下 SEV -SNP 并未实现客户机与主机之间的 BTB 隔离；而在 Zen 5 架构中，我们观察到预测结果在主机与 SEV -SNP 客户机之间实现了有效隔离。遗憾的是，我们目前缺乏 Zen 4 服务器来验证这一隔离效果。虽然 Zen 1和Zen 2不支持 SEV -SNP，但在使用 SEV 或 SEV -ES等较弱技术的场景中，它们可能存在安全漏洞。

> **观察结果 O7：** AMD Zen 3 存在 H → G| SEV -SNP 攻击漏洞。

SEV -SNP为访客提供了多种可选保护机制，我们将在第9.3节中详细讨论。我们的分析表明，早期的、eIBRS之前的Intel微架构对$vBTI_{HS\to GU}$、$vBTI_{HU\to GU}$、$vBTI\text{-}SMT_{HS\to GU}$以及$vBTISMT_{HU\to GU}$这些基本操作存在漏洞；但这些系统并不支持 TDX 。

6.3. 宾客之间的攻击

我们分析的第三个威胁模型是 $TM_{G1\to G2}$。在此模型中，恶意客户端会攻击在同一主机系统上运行的另一个客户端。

(S5) G_1 → G_2 | 客户进程。 在此场景中，恶意客户进程会针对另一个客户进程的用户进程发起攻击，这凸显了不可信虚拟机之间共存的风险。这表明不同客户进程之间缺乏 BTB 隔离。与先前场景类似，我们区分了同线程和跨 SMT 两种情况：在同线程情况下，受害者会在攻击者线程被抢占后，于同一线程上执行；此外还要求虚拟机切换点缺乏 BTB 清理机制——不过Linux内核通过虚拟机切换点上的 IBPB 功能对此进行了缓解；而跨 SMT 情况则需要 BTB 状态在兄弟线程间共享。根据我们在第5.2节中的逆向工程分析，Zen 1和Coffee Lake架构均缺乏 SMT 线程隔离机制，因此易受G_1 → G_2|客户进程攻击的影响。

> **观察结果 O8。** AMD Zen 1 和 Intel Coffee Lake 存在易受 G_1 → G_2| Guest Process 攻击的漏洞。

然而，实际应用仍面临诸多挑战。在公共云主机上，攻击者与客户虚拟机之间通常缺乏直接的交互通道，也无法利用共享内存实现侧信道攻击。

6.4. 摘要

表3概述了各类攻击场景，总结了其前提条件、目标对象及易受攻击的微架构。我们的分析表明，vBTI原语可在第3节介绍的三种威胁模型中均导致具体的安全漏洞。这些攻击包括：恶意客户机对主机发起的攻击（$TM_{G\to H}$）、恶意主机对客户机发起的攻击（$TM_{H\to G}$），甚至两个客户机之间的攻击（$TM_{G1\to G2}$）。

为证明vBTI攻击的可行性，我们推出了 VMS CAPE——一种针对G → H|$Hypervisor_{user}$场景的端到端漏洞利用工具。该场景代表了

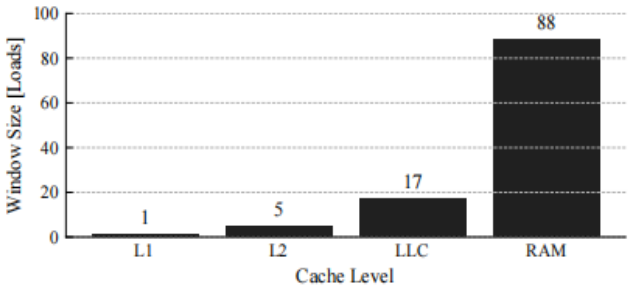


Figure 6. Speculation window size on Zen 4 as the maximum number of executed dependent loads with the pointer to the indirect branch destination residing in the respective cache level.

这对云服务提供商而言尤为严重，因为虚拟机监控程序$_{\text{用户}}$内存的泄露直接针对其基础设施。

7. 扩大投机窗口

我们的目标是在 G → H| Hypervisor $_{\text{user}}$ 场景下构建一个端到端的内存泄漏漏洞利用方案，针对最新的 AMD Zen 4 和 Zen 5 系统。在构建该漏洞利用方案的过程中，我们发现受害分支的推测窗口过短。首先，我们将阐述如何评估推测窗口的大小；随后，讨论如何通过缓存驱逐机制来扩大该窗口。

预测窗口大小。 为测量预测窗口的大小，我们设计了以下实验：使用一个从内存获取目标地址的间接分支指令，从而使 BPU 能够对目标地址进行预测和推测。预测窗口的大小取决于目标地址在架构层面的解析速度，而该速度又取决于存储间接分支目标地址的缓存层级。我们可以通过在预测过程中连续执行n次依赖性加载操作，并检查最后一次加载是否命中缓存来测量预测窗口的大小；同时确保除最后一次加载外的所有加载操作均位于L1缓存中。对于每个$n \in [1, 127]$，我们重复实验4096次，并调整存储间接分支目标地址的缓存层级。

如图6所示，当目标地址位于L1层级时，在推测窗口期间仅执行一次依赖性加载操作。对于基于F LUSH+R ELOAD的侧信道攻击而言，这种推测窗口长度是不足的，因为该攻击至少需要两次依赖性加载操作。虽然我们在L2层级的研究结果表明存在足够大的推测窗口以实现完美的推测装置，但某些攻击仍需要更长的推测窗口。这可能是由于先前指令产生的额外竞争延缓了披露装置的运行速度，或是因为受害软件在指令流中更早阶段就开始加载目标地址（例如用于验证其是否为有效指针）。这些因素会缩短披露装置可用的推测时间，迫使攻击者扩大整体推测窗口的范围。

可通过清除存储间接分支目标地址的内存来延长推测窗口。

TABLE 4. CACHE DETAILS ON AMD ZEN 3, ZEN 4 AND ZEN 5. THE SIZES MAY VARY BETWEEN PROCESSORS OF THE SAME GENERATION.

Core	L1			L2			*Last Level Cache* (LLC)		
	Size	Sets	Ways	Size	Sets	Ways	Size	Sets	Ways
Zen 3	32KiB	64	8	512KiB	2048	8	32MiB	32768	16
Zen 4	32KiB	64	8	1MiB	2048	8	32MiB	32768	16
Zen 5	48KiB	64	12	1MiB	1024	16	32MiB	32768	16

从缓存中读取数据会导致受害分支的解析速度变慢[1]。然而，虚拟机内的攻击者无法直接清除目标指针，因为支持该指针的物理内存是分配给主机进程的，虚拟机无法访问。因此，攻击者必须将目标指针从缓存层次结构中驱逐出去。需要强调的是，驱逐操作应仅针对必要的缓存集，以避免在推测阶段拖慢整个受害程序的执行速度。

7.1. 在 AMD Zen 4 和 Zen 5 上的 LLC 强制清除

Wang等人最近的研究[26] 我们验证了在AMD Zen 3架构上构建LLC置换集的可行性。基于他们的研究成果，我们首次为AMD Zen 4和Zen 5架构实现了LLC置换机制。为实现这一目标，我们遍历了缓存层次结构的所有层级，识别出与Zen 3架构的差异，并逐步学会置换更大层级的缓存单元，从而延长推测窗口时间。随后，我们展示了如何将该置换集构建流程移植至虚拟机环境，并利用 XOR 索引技术显著提升流程效率与可靠性。表4列出了所评估处理器的相关缓存架构参数。为简洁起见，本文首先详细阐述Zen 4架构，再总结其与Zen 5架构的差异。

在逆向工程中，我们采用1GB分页机制以确保虚拟地址与物理地址之间的位匹配。此外，我们还利用了AMD精确的 APERF 计时器，该计时器可通过rdpru指令访问[27]。默认情况下，1GB分页机制及精确计数器均不对客户机开放。因此，我们仅在逆向工程过程中使用这些功能，而不用于第8节所述的端到端漏洞利用方案。

缓存访问延迟。 我们首先对每个缓存层级的内存访问时序函数进行校准，从而确定内存访问是由 L1、L2、LLC 还是主内存提供的。为计算延迟，我们按以下步骤进行：从包含 N 个缓存行的内存区域开始，以伪随机顺序访问所有缓存行并记录访问时间；随后逐步增大 N 值重复此过程。图 7 展示了随着内存区域大小 N 增大而变化的中位访问时间曲线，其中平台期对应的延迟分别表示 L1、L2、LLC 和 RAM 的访问延迟。

L1 驱逐机制。 根据我们先前的缓存访问测量结果，可以确认 Zen 4 的 L1 索引方案与 Zen 3 相同。通过访问 8 个不同的缓存行（对应 L1 的各路），即可驱逐任何 L1 条目；这些缓存行均共享相同的位 [6:11]。

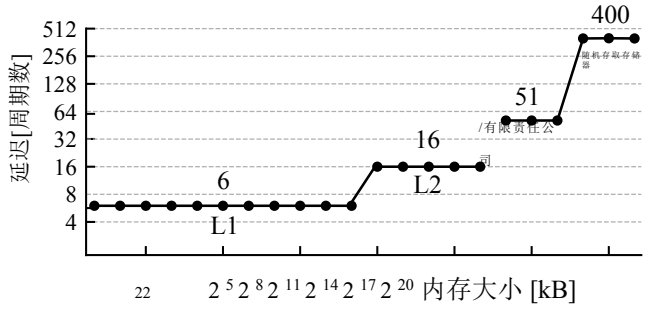


图7. Zen 4架构下不同大小内存区域的中位内存访问延迟，数据以对数坐标轴呈现。平台区内的延迟读数分别表示L1缓存、L2缓存、LLC缓存及RAM的延迟值。

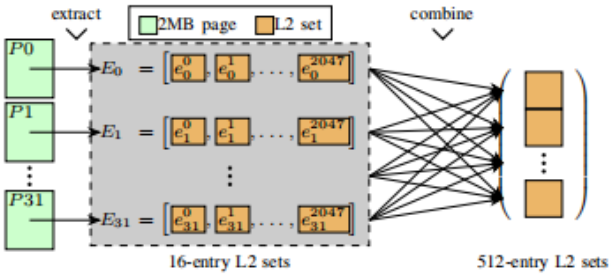


图8. 针对基于虚拟机的攻击者的LLC缓存驱逐集构建策略。首先，我们*提取*不同2MB页面中包含的L2驱逐集；其次，我们将来自不同2MB页面的L2驱逐集*合并*。

L2驱逐集。 假设L2索引的设计方式使得2MB大页内的内存均匀分布在所有L2缓存集中。因此，需要在2MB页面边界以下至少11位索引位才能映射到全部2048个L2缓存集。基于先前对Zen 3架构缓存逆向工程的研究[26]，我们进一步假设2MB页面边界以下的部分高位未用于L2索引。鉴于2MB页面中共有21位地址位，其中11位用于索引、6位用于64字节缓存行内的偏移量，我们推测最高4位未被用于缓存索引。这将导致每个L2缓存集包含16个条目，我们认为这足以实现对全部8路缓存的驱逐操作。

我们通过以下方法验证上述假设：首先随机选取一个2MB的大页面，根据[6:16]位的数值将该大页面内所有缓存行对齐地址划分为若干集合；随后通过测量重复访问每个集合内所有条目的平均延迟时间，确认这些生成的集合均具备自驱逐特性；进一步通过测量从每个集合中重复访问其半数地址的延迟时间，确保任意两个生成集合之间不存在相互驱逐现象。基于实验结果，我们得出结论：AMD Zen 4架构中的L2缓存索引机制并未使用[17:20]位。

观察结果 O9： 在 AMD Zen 4 架构的 L2 缓存索引过程中，比特位 [17:20] 未被使用。

AMD Zen 4架构的二级缓存（L2）具备8路替换通道，根据*最近最少使用（LRU）*替换策略，理论上应至少需要8个地址才能完成有效置换。然而实验表明，要可靠地将目标地址从L2缓存中驱逐出去，实际至少需要12个地址。我们认为这种现象源于该替换策略比 LRU 更为复杂。

LLC驱逐集。基于AMD Zen 4架构的LLC是L2缓存的非包含式受害者缓存。因此，若要将缓存行插入LLC进行驱逐，这些缓存行必须先从L2中被驱逐出去。根据给定的L2驱逐集，我们可以识别出同一1GB巨大页面内的所有地址。

通过检测这些地址是否被该驱逐集清除，即可确定它们与该驱逐集的对应关系。假设地址在1GB大页内的2048个L2缓存集中均匀分布，我们预计会有8192个地址映射到同一L2缓存集——这一结果已通过实验得到验证。进一步研究证实，其中前512个地址已能为同一L2缓存集内的任意地址提供可靠的LLC驱逐机制。虽然这并非最小规模的驱逐集，但其效率足以满足攻击需求，并显著延长了推测窗口时长，如图6（RAM）所示。

Zen 5： 由于 L1 和 L2 缓存中增加了更多存储单元，缓存驱逐集的大小相应增加。与 Zen 4 相比，Zen 5 减少了 L2 集的数量，使其再次与 Zen 3 相匹配。因此，我们发现 2 MB 分页中的相同位段（即 [16:20]）仍像在 Zen 3 中一样未被使用。最后，与 Zen 4 相比，Zen 5 需要两倍数量的地址，并且必须对每个地址进行两次访问，才能可靠地扩大推测窗口。

7.2. 从虚拟机中退出

我们的LLC驱逐机制使攻击得以成功实施，但该机制采用了精确计数器和1GB页面大小——这两项功能默认情况下对 QEMU 客户端不可用。此外，通过暴力破解新地址集合成员来增大每个L2驱逐集的规模既耗时又易受噪声干扰。首先，我们通过同时测量驱逐集中所有条目的访问时间而非分别测量，从而克服了计时器限制以增强信号强度；其次，我们提出了一种显著更高效的驱逐集生成方法，且无需依赖1GB巨型页面。

我们注意到，Linux默认采用透明的2MB大页机制来支持虚拟机页面，以此提升性能。因此，我们在虚拟机内部构建基于2MB页面的L2缓存集的方法仍然适用。此外，我们认为，在拥有大量此类2MB页面的情况下，可以如图8所示为每个页面*提取*对应的L2缓存集。虽然这为我们提供了足够的地址空间用于LLC置换策略，但仍需要找到一种高效的方法来*整合*来自不同页面生成的L2缓存集。

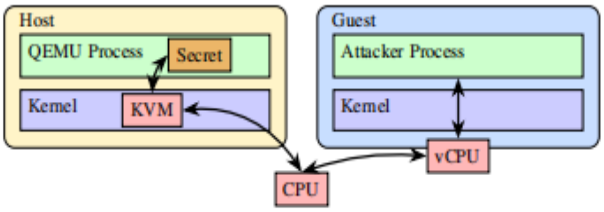


图9。基于 KVM 的客户机运行于支持硬件虚拟化的CPU上，由 QEMU 管理。

令$_{eij}$表示源自页面j的第i个驱逐集，其中i由L2集索引位[6:16]确定。令$E_j = \{eij \mid i \in [0, 2047]\}$表示页面$j$的所有驱逐集集合。我们的目标是合并$E_j$（$j$）中的所有L2集。$\neq 0$）映射到$E_0$的L2集合中。现在假设对于某个$j$、$a$和$b$，集合$_{eaj}$与集合$eb_0$发生碰撞。我们需要高效地确定其将与哪个集合$_{ecj}$发生碰撞。我们假设L2和LLC采用了基于 XOR 的索引函数，该函数可按以下方式计算出发生碰撞的集合ed_0：

$$d = a \oplus b \oplus c$$

采用这种方法，可以为每个E_j（j）中的单个元素找到匹配的L2集合。$\neq 0$）足以确定所有其他元素的冲突驱逐集。Zen 3之前的方案[26]需要通过逐一采样地址并测试*每个*地址是否被驱逐来增大初始L2集的规模。我们的新方法仅需对该过程进行*一次*操作，利用2MB页面内的整个第一L2集，从而无需进一步测试即可为同一页面中的所有其他32752个地址分配各自的驱逐集。此外，该方法可靠性极高：在100次运行中首次尝试即实现LLC驱逐集创建的100%成功率，而先前研究[26]在使用1GB页面时对Zen 3的报告成功率约为60%。

观察结果O10： 基于 XOR 的缓存索引功能能够高效地将来自不同2MB页面的L2数据集合并为更大的L2数据集。

随着投机窗口的成功延长，我们现在着手构建 VMS CAPE。

8. VMSCAPE

我们在第5节介绍的跨域实验表明，所有AMD Zen CPU和Intel Coffee Lake CPU均易受vBTI $_{GU \to HU}$攻击模式的影响。本节我们将介绍 VMS CAPE——一种作为恶意客户机泄露虚拟机监控程序密钥的端到端攻击工具。图9展示了相关组件架构。我们的攻击目标是 QEMU（Linux生态系统中工业领域常用的虚拟机监控程序$_{用户}$[15]）。然而，威胁模型（参见第3节）或攻击设计中的任何假设，均无法从根本上阻止使用稍作修改的技术对其他虚拟机监控程序$_{用户}$实施攻击。我们已在搭载Linux系统的AMD Zen 5服务器CPU上执行了该攻击。

该软件包含内核版本6.8.0-85Generic及Ubuntu 25.10软件包，内置最新 QEMU v10.0.2。我们进一步验证了 VMS CAPE能够在AMD Zen 4系统上成功泄露信息。

在第7节中，我们解决了该攻击所引发的推测窗口问题。然而，要实现端到端的漏洞利用，还需解决另外两个挑战：

- 首先，我们需要确定一个合适的漏洞利用链，该链由侧信道、推测装置和披露装置组成（详见第8.1节）。
- 其次，我们需要对 QEMU 的*地址空间布局随机化*（ASLR）进行去随机化处理，以确定受害分支地址及重载缓冲区地址（详见第8.2节）。

最后，我们将提出的解决方案与推测窗口延长技术相结合，构建了一种针对 QEMU 的任意内存泄漏攻击方法（详见第8.3节）。

8.1. 漏洞利用链

我们的目标是在 QEMU 未修改的代码中发现一条漏洞利用链。要构建这样的链，我们需要在 QEMU 二进制文件中找到合适的侧信道、推测装置和披露装置。

侧信道。 我们首先定义用于泄露主机机密的侧信道。通过研究，我们发现 QEMU 将所有客户机内存映射到其虚拟地址空间；主机与客户机之间的共享内存使得F LUSH+R ELOAD成为一种侧信道[20]。

观察结果 O11。 QEMU 将所有客户机内存映射到 Hypervisor $_{用户}$ 虚拟地址空间，从而实现 F LUSH +R ELOAD。

F LUSH+R ELOAD通过检测近期被访问过的内存地址（与受害者共享的）来泄露机密信息。作为侧信道使用的共享内存被称为*重载缓冲区*。

推测装置。 我们的攻击链需要触发推测操作，而vBTI原语通过间接调用或间接跳转来实现这一点。我们在能够对寄存器进行一定程度控制的位置寻找此类推测装置，从而影响被推测目标的执行过程。通过使用F LUSH+R ELOAD，我们旨在控制两个寄存器：一个存储秘密指针，另一个存储重载缓冲区地址。此外，所需的推测装置可以被重复、可靠且高频地触发，从而实现高带宽泄漏。

在管理基于 KVM 的虚拟机时， QEMU 主要负责I/O操作和设备仿真。因此， QEMU 的实际代码量相对较小。我们仅将搜索范围限定在通用I/O处理代码上，避免针对特定设备仿真进行分析，从而不对威胁模型提出额外假设。

如图所示，我们在 QEMU 的*内存映射I/O*（MMIO）写入处理中找到了合适的受害者分支。

```
1  static MemTxResult memory_region_write_accessor(
2                      MemoryRegion *mr,
3                      hwaddr addr,
4                      uint64_t *value,
5                      unsigned size,
6                      signed shift,
7                      uint64_t mask,
8                      MemTxAttrs attrs) {
9      uint64_t tmp = memory_region_shift_write_access(
10                    value, shift, mask);
11     // [...]
12     mr->ops->write(mr->opaque, addr, tmp, size);
13     return MEMTX_OK;
14 }
```

Listing 1. Extract from QEMU's `system/memory.c` source file, showing the victim branch on line 12, with the attacker-controlled value in `tmp`.

如清单1所示，攻击者能够控制tmp中的值，该值对应于MMIO 操作中写入的数据。虽然攻击者在函数调用时仅直接控制单个64位寄存器（*RDX*），但动态分析表明，第二个寄存器（*R12*）中也保留了先前使用的相同数值。此外，我们可以通过共享内存传递额外值，但这需要在披露机制中执行一次额外的加载操作。

> **观测结果O12：** MMIO 实现了客户机与虚拟机监控程序之间可重复、可靠且高频次的寄存器控制交互。

需要注意的是，虽然通用 MMIO 支持8字节写入操作，但被写入的设备也必须支持这种粒度的写入。在我们的主要攻击方案中，我们采用了 QEMU 虚拟机默认提供的hpet高精度事件计时器。

信息泄露装置。 为了从攻击对象处获取机密信息，我们需要一种能够利用 F LUSH+R ELOAD 的信息泄露装置。我们基于 Wikner 和 Razavi [2] 提出的信息泄露装置扫描器进行改进：由于原始扫描器无法在 QEMU 二进制文件中检测到所需装置，我们通过优化针对使用不同污染状态基址寄存器和索引寄存器的内存指令的污染逻辑来扩展该扫描器；同时新增了链式装置搜索功能——将两个独立装置组合起来，其中第一个装置以间接调用结尾，可通过训练使其错误预测第二个装置的行为。这些改进使我们能够成功定位目标信息泄露装置。清单2展示了本攻击中使用的最终装置链。

该装置不会对密钥应用任何乘数，这可能对 F LUSH +R ELOAD 造成问题。然而，先前的研究表明，我们可以通过采用调整重载缓冲区基址的技术，并检查密钥是否最终位于不同的缓存行中，来克服这一限制 [2]。

分支冲突。 当到达 QEMU 中的目标分支时，控制流刚刚经过一个 VMEXIT 指令后执行了KVM Exit操作。因此，分支历史记录会受到客户机、 KVM 和 QEMU 中各分支的影响。

```
1 // first part of the gadget chain
2 0xac334e mov rdi, qword ptr [r12 + 0x2b58]
3 0xac3356 call qword ptr [r12 + 0x2b50]
4
5 // second part of the gadget chain
6 0x8260b5 add cl, byte ptr [rdi]
7 0x8260b7 test dword ptr [rdx + rcx], esp
```

Listing 2. Disclosure gadget in two parts. Line 2 loads the secret pointer, line 6 retrieves one byte of the secret and line 7 accesses the reload buffer.

具体取决于历史缓冲区的大小。由于AMD基于历史记录的分支预测优先于静态 BTB 预测，因此需要覆盖或作废针对注入目标的历史预测结果才能获得有效观测数据。在训练过程中精确复现历史记录虽可行，但难度较大。根据AMD官方文档[18]所述：28]仅对那些接触过多个攻击目标的间接分支才会考虑其历史记录。通过在训练前使用 IBPB 清空 BPU，恶意攻击者可确保受害分支仅观察到训练目标。因此，无需参考最后两个相对容易复制的分支之外的历史数据，即可错误预测该分支。尽管允许访问 IBPB 的攻击者看似是一个强有力的假设，但实际上这正是预期中的情况——因为攻击者本应能够自行实现域隔离。

> **观察结果O13：** 客户端软件可利用对缓解控制措施的访问权限，影响主机软件可使用的分支预测条目。

然而，AMD Zen 4上的Zen 5及后续微代码更新修改了 IBPB，使其同时使 BTB 中的直接分支类型信息失效，从而有助于缓解*推测性返回栈溢出*（SRSO）[6]。29]这无意中缩小了我们攻击的推测窗口大小——推测是因为第一个受害者指针加载与受害者分支之间的直接分支会减慢推测执行速度。因此，我们必须在不使用 IBPB 的情况下，让 BTB 忘记受害者分支实际上是一个具有多个目标的间接分支。对于AMD Zen 4和Zen 5架构，我们可以通过以下方式实现：将受害者 BTB 条目替换为虚假的直接分支预测。(1)用直接跳转指令替换训练分支；(2)执行训练流程；(3)将原始训练分支写回内存。此方法符合AMD针对Retbleed漏洞制定的Jmp2Ret缓解方案[30]。

如今，我们能够从 QEMU 中可靠且重复地劫持受害分支，将临时执行流程重定向至我们的披露机制。

8.2. 打破 ASLR

Linux默认使用 ASLR 机制对用户空间代码、堆和栈的虚拟地址进行内存随机化。要实施成功的攻击，我们需要确定推测装置（speculationgadget）和泄露装置（disclosuregadget）的具体位置。

该虚拟地址是 QEMU 映射的虚拟机内存地址，其中存储着我们的重载缓冲区。

部分代码 ASLR 去随机化。 Linux将 ASLR 实现为相对于程序非 ASLR 基础地址的随机偏移量。最大偏移量是一个可配置参数，其默认值由Linux动态确定。目标系统采用默认的最大偏移量为2^{32}。

与先前的研究类似，我们的目标是通过检查受害分支的所有可能位置[8]来确定推测装置的地址。31]他们在攻击者情境下的某个假设受害者位置V_i处训练分支B_i，触发受害者指令的执行，并测量对B_i的预测是否被清除。该方法在我们的案例中不适用，主要基于两个原因：首先，与英特尔处理器不同，AMD Zen 4和Zen 5架构上的 BTB 并非每次预测错误都会覆盖目标地址，而是将新目标地址插入ITA中，同时保留 BTB 中的原有目标地址；其次，我们需要分别检查多达约2^{32}个位置，这会耗费极长时间。

我们通过反转攻击者与受害者的角色（如先前研究[7]所提出）来优化上述方法。给定假设的受害分支位置V_i，我们可以计算出对应的靶地址T_i。因此，我们首先触发受害者训练分支预测器，然后在攻击者域的V_i位置对某个非信号目标执行匹配分支。通过在攻击者域中假设的受害目标位置T_i额外部署一个信号装置，即可检测碰撞事件。采用新方法时，仅需一次受害者分支执行和一次信号相关的重载缓冲区重载操作，即可检查多个潜在受害者位置（例如1024个位置）。一旦观测到信号，我们就能确定哪个位置是正确的。

结果。 虽然该方法能在Zen 4架构上确定单个受害者的位置——直接破解代码 ASLR，但在Zen 5架构上我们获得了多个候选目标。由于所有候选目标共享相同的低24位数据，我们认为这是由别名效应以及 BTB 仅提供部分目标所致（与Intel方案类似[8]）。17]虽然这种方法能揭示受害者分支地址的低24位信息，但仅对代码 ASLR 进行了部分去随机化处理（该代码负责生成页面偏移位上方的32位随机值）。我们发现：当仅使用匹配的低24位数据训练预测器时，其输出结果同样仅包含目标地址的低24位信息。因此，由于代码 ASLR 仅经过部分去随机化处理，我们只能推测目标地址与受害者分支地址之间的偏移量在24位以内。然而，我们所采用的披露装置链位于距离受害者分支地址更远的位置（超过$2^{\wedge}4$倍），这要求我们必须完全破解代码 ASLR 才能获取完整信息。

定位重载缓冲区。在完全实现代码 ASLR 之前，我们需要确定 QEMU 为攻击者虚拟机使用的内存所映射的虚拟地址，该地址用于存储重载缓冲区。我们还需要通过F LUSH+R ELOAD最终向重载缓冲区泄漏任意数据。与先前研究[2]类似，我们采用暴力搜索法确定重载缓冲区的位置，并进行了部分去随机化处理。

基于先前的研究结果，我们可以推测目标地址位于受害者分支指令产生的24位偏移范围内。在该区域内，我们发现了一个能够加载攻击者指定内存地址的机制。值得注意的是，正如先前观察到的那样，Linux与 QEMU 系统会将虚拟机页面映射为2MB的大页面。因此，通过将重载缓冲区映射为虚拟机内的2MB大页面，我们可以显著减少潜在的重载缓冲区位置数量。此外，由于搜索重载缓冲区仅需在推测窗口内执行一次加载操作，这意味着我们需要相应延长此步骤的推测窗口长度。

结果。 经过100次重复实验，我们能够以100%的成功率实现重载缓冲区位置的去随机化处理，在Zen 5和Zen 4处理器上的耗时中位数分别为46秒和83秒。

完全破解代码 ASLR 。 在定位到重载缓冲区后，我们现在可以彻底消除代码 ASLR 的随机性并确定推测指令getdget的位置。这一过程同时揭示了泄露装置的位置——因为 ASLR 仅对可执行文件的基地址进行随机化处理，而保留了偏移量信息。对于Zen 4架构而言，此额外步骤并非必需，因为其 BTB 架构未表现出别名现象，因此可直接获取受害者内存位置。

在受害者分支处，寄存器*RAX*包含指向某个代码位置的指针。掌握该值可让我们计算出完整的受害者分支地址。我们可以通过位于受害者分支24位偏移量内的一个额外机制来泄露此指针：该机制将攻击者控制的寄存器*RDX*与*RAX*相加，并加载所得地址。通过暴力尝试*RDX*的不同取值，我们可泄露*RAX*的地址，从而在重载缓冲区中记录命中结果。

结果。 在Zen 5架构上，完全破解 ASLR 码的中位耗时为12秒，准确率为75%（未计入定位重载缓冲区所需时间）；而在Zen 4架构上，由于第一步已能确定受害者分支的位置，破解 ASLR 码耗时238秒，准确率为70%。时间差异源于：在Zen 5架构的代码 ASLR 破解第一步中，只需从2^{12}个候选地址中找到一个有效目标；而Zen 4架构则需检查全部2^{32}个可能地址才能定位正确位置。Zen 4架构第一步所需的搜索空间更大，导致其耗时显著长于Zen 5架构所需的第二步——后者通过泄露 ASLR 熵的剩余20位信息来实现破解。

8.3. 任意内存泄漏

我们现在已具备构建任意内存泄漏攻击所需的所有组件：漏洞利用链、ASLR破解原语，以及一种能够提供足够大推测窗口的可靠方法。

我们的 VMS CAPE攻击流程如下：首先，我们利用 ASLR 破坏技术定位目标分支；确定相关gadget后，即可获取 QEMU 重载缓冲区的虚拟地址。接着构建LLC驱逐集，并通过测试哪个驱逐集能使依赖性加载操作产生缓存跟踪痕迹来确定正确集合；最后结合长推测窗口机制，利用这些gadget泄露 QEMU 的内部数据结构。

找到秘密数据的地址，然后泄露该秘密数据。

结果。 对Zen 5芯片实施完整的端到端攻击所需中位时间为102秒，可泄露一个4 *KiB*的 QEMU 密钥对象（以磁盘加密/解密密钥为例）。在总攻击时间内，需要58秒即可定位受害分支和重载缓冲区。获取这些信息后，我们的漏洞利用工具能以154字节/*秒*的速度泄露任意 QEMU 内存，字节精度达99.85%。该攻击的整体成功率为68%。

9. 讨论

我们讨论了针对 VMS CAPE攻击的NUMA相关考量（第9.1节）、我们在首次负责任披露后发现的其他vBTI变体（第9.2节），以及针对vBTI原语的缓解策略（第9.3节）。

9.1. NUMA考虑事项

我们发现，在采用NUMA技术的Zen 5系统中， VMS CAPE无法访问某些内存页面。经深入排查，发现Linux的NUMA均衡机制[32]会定期将页面标记为“不存在”，以追踪这些页面是否被同一NUMA节点内的CPU核心频繁访问。被标记为“不存在”的页面无法通过推测执行机制进行访问——这实际上阻止了 VMS CAPE对虚拟机监控程序不常访问的内存页面进行操作。然而，云虚拟化平台和提供商通常会在同一NUMA节点上为虚拟机分配物理内存以提升性能[33]。34], [35]在这样的实际应用场景中，Linux内核建议关闭NUMA平衡功能以减少不必要的跟踪开销[32]。

9.2. BHI 变体

英特尔对经典 Spectre- BTI [1]的应对方案是eIBRS——一种能有效隔离 BTB 中用户与管理员权限条目的硬件缓解措施。然而，*分支历史注入*（BHI）[3]通过利用用户域与管理员域之间缺乏 BHB 隔离的漏洞，重新激活了跨权限的Spectre攻击。如第5节所示，eIBRS在虚拟化环境中成功实现了 BTB 隔离。但与 BHI 类似，eIBRS本身并不能完全阻止客户机对分支历史进行（部分）控制。虽然针对HS的攻击对此问题有所缓解[5]，36]我们认为，该漏洞（HU）目前尚无缓解方案。尽管我们未对此进行核实，但英特尔已确认情况确实如此，这表明近期生产的英特尔CPU可能易受*基于虚拟化的Spectre- BHI（vBHI）攻击原理*的影响。

9.3. 缓解；减轻

如第5节所述，我们的逆向工程揭示了近期多种微架构中存在多个vBTI基本单元。本节将探讨相应的缓解策略。需防止此类原始漏洞，首先是从 IBPB -on- VMEXIT 开始——该漏洞正是已部署的 VMS CAPE补丁的基础。

IBPB -on- VMEXIT 。 该缓解措施通过在每个 VMEXIT 操作时使用 IBPB 刷新 BPU 状态，从而在TM $_{G\rightarrow H}$模型中保护主机。因此，它确保了每次从客户机到主机的转换过程中实现客户机与主机之间的 BTB 隔离。然而，基于 IBPB 的缓解措施通常会带来显著的性能开销[2]。6]当应用于 VMEXIT 等快速路径时，这一问题尤为突出。

为了量化这一开销，我们使用UnixBench测试套件[37]对缓解措施进行了基准测试，具体方法遵循了先前研究[2]的方法论。6], [8]我们在一个配备4GB内存和两个核心的虚拟机环境中，以多线程模式运行工作负载。在Zen 5处理器上，性能开销（基于五次重复测试）的几何平均值为57%。

Linux内核开发者基于IBPBon- VMEXIT 方案制定了缓解措施，并通过仅在伴随客户机执行的KVM_EXIT操作时触发 IBPB 来对其进行优化。由于并非所有 VMEXIT 都会导致KVMEXIT及后续用户空间执行，此改进将 IBPB 移出快速路径，同时保留其安全保证。他们将该缓解措施称为 *“退出用户空间前触发 IBPB ”* 。该补丁适用于所有受影响系统，包括AMD Zen 5架构处理器以及Lunar Lake和Granite Rapids等最新款Intel CPU。

除了验证补丁的有效性外，我们还在相同测试环境中对其性能开销进行了基准测试。UnixBench结果显示，在Zen 4架构上仅产生1%的微小开销。然而，作为面向计算性能的基准测试工具，UnixBench导致用户空间退出次数相对较少。为了评估那些会频繁触发用户空间退出的工作负载，我们使用fio[38]生成磁盘I/O数据——通过在虚拟化磁盘上随机读写10GB数据并重复执行10次操作。在Zen 4架构下，优化后的缓解方案开销达到51%，接近 IBPB -on- KVM_EXIT方案57%的开销水平。当使用由主机内核（vhost）或直通设备管理的设备时，用户空间退出会被降至最低。这些结果表明：该缓解措施对性能的影响既取决于工作负载类型，也受 QEMU 配置影响；但在大多数实际应用场景中，其影响预计仍处于次要地位。

Retpoline。 Retpoline 是一种经典的基于软件的 SpectreBTI 缓解方案，它通过使用不同的预测器[39]预测出的其他间接分支（ret），来替换某些间接分支（jmp 和 call）。在未受到早期针对返回点攻击影响的系统上，使用 retpoline 编译 Hypervisor$_{用户}$程序可使其免受 G → H|Hypervisor$_{用户}$场景下的攻击者侵害[6]。虽然该方案无法保护涉及 G → H|Host Process 的其他主机用户进程，但其产生的性能开销低于 IBPB -on- VMEXIT 。使用 UnixBench 工作负载对 retpoline 的性能评估显示开销为 3.9%。根据威胁模型的不同，将 retpoline 与使用 PR_SETSpecificationULATION_CTRL prctl 实现的选择性主机进程隔离相结合，可作为高开销 IBPB -on- VMEXIT 的替代策略。我们的研究表明

目前流行的开源Hypervisor用户均未部署retpoline[13]。40], [41], [42]该缓解措施仅适用于不受Phantom推测影响的CPU[43]，例如Zen 5。

保护 SEV -SNP客户机。 虽然受vBTI原语影响的英特尔系统不支持 TDX，但Zen 3至Zen 5架构上的 SVM 为启用 SEV -SNP的客户机提供了多种可选保护机制[18]。28](1) *分支目标缓冲区隔离* 技术可防止宿主域外部的执行操作影响宿主执行环境内的分支决策。(2) *入口处的间接分支预测屏障* 机制迫使CPU在每次 VMRUN 指令执行时都必须发出 IBPB 指令。(3) *SMT 保护机制* 会强制使兄弟 SMT 线程处于空闲状态，而宿主操作则在另一个兄弟线程上执行。我们在第5节 的实验表明，Zen 5架构默认已为 SEV -SNP宿主提供足够的隔离能力。因此，此类防护措施在早期架构的CPU上同样适用。

保护 SMT 。 STIBP 是一种硬件缓解措施，可隔离 SMT 核心的 BPU 。我们发现大多数系统默认启用 STIBP 功能，从而防范vBTI- SMT 原语的影响。然而，在某些CPU（如Coffee Lake）中， STIBP 并非*始终处于开启状态* 。对于不支持 STIBP 的系统（例如Zen 1架构），唯一可行方案是完全禁用 SMT ，但这会导致性能显著下降[2]。44].

基于硬件的隔离。 最终，vBTI原语的根本原因在于主机域与客户域之间缺乏 BPU 状态隔离。Zen 5处理器引入了特权域标记机制以区分用户域和管理员域；采用类似的标记机制来区分主机域与客户域可有效防止vBTI原语的发生，这一方案应纳入Zen微架构的后续版本更新中。

10. 相关工作

关于微架构安全性的研究已有数十年的深厚积淀，其研究范围始于时序侧信道分析，尤其聚焦于缓存领域[45]。46]. Aciic ¸mez等人[47], [48] 发表了类似的研究，但其研究对象为早期分支预测因子。Evtyushkin等人[31] 后续研究表明，现代BTB技术同样可用于破解 ASLR 。Ge等人[49] 本文提出了一种针对此类（及其他多种）可通过共享资源与上下文利用的时序侧信道的分类方法。

Spectre。 Kocher等人[1] 首次提出了由分支预测错误引发的瞬态执行攻击。随后又出现了大量其他类型的瞬态执行攻击[50]、[51], [52],[53], [54], [55], [56], [57] Canella等人[58] 对这类攻击进行了评估与分类。根据受害者的防护领域，我们将先前提出的推测性执行攻击分为三类：(1)虚拟机监控程序攻击；(2)管理程序攻击；(3)用户空间攻击。

(1) 虚拟机监控程序攻击。 关于 Spectre 的原始研究[1]，以及后续的研究[5]，8]我们针对虚拟机监控程序软件中经过修改的监督组件进行了概念验证攻击。谷歌研究人员基于Inception[6]，部分公开了针对 KVM 的攻击方案[59]。我们的研究工作.....

相比之下，这是首个真正的端到端攻击案例：恶意 KVM 客体专门针对虚拟机监控程序的*用户空间*组件发起攻击。

(2) 监督者攻击。 监督者模式自早期起便是 BTI 漏洞利用的热门目标[1]。尽管CPU厂商和操作系统开发者争相开发防护措施，研究人员仍不断发现新防御机制中的漏洞。Wikner与Razavi通过推翻预测器训练的基本假设，在Retbleed[2]、Phantom[43]及Inception[6]中规避了retpole线机制与硬件级防护手段。 BHI 及其后续研究反复证明：即便存在多种防范措施，同时模式训练仍是可行的[3]。4], [5]这打破了基于交叉权限 BTI 缓解措施所建立的假设。

(3) 用户空间攻击。 已有多种针对用户空间的攻击方法被提出，尤其是针对浏览器沙箱技术的攻击。Ragab等人[60] 广义机器清除被确认为推测性执行的来源，并发现了他们在浏览器中利用的新型机器清除诱因。Ret2spec [61] 和 Spectre Returns [21] 均通过探索*返回栈缓冲区*（RSB）预测机制，在多种场景下攻击用户空间受害者。Spring [62] 随后证明，尽管存在各种系统级和浏览器级防护措施， RSB 预测仍可被利用。更近期的攻击还针对其他预测器，同样实现跨沙箱边界的信息泄露[63]，[64]。Wikner与Razavi [7] 首次完整展示了端到端的跨进程攻击方案。与以往仅针对用户空间的攻击不同，我们识别并利用了不仅能跨上下文操作，还能同时跨越权限与虚拟化边界的原语。

11. 结论

我们提出了 VMS CAPE——这是首个在 QEMU 平台上由恶意 KVM 客户机发起的实际SpectreBTI攻击，可在AMD Zen 5处理器上以154字节/*秒*的速度泄漏内存。与以往由恶意客户机发起的Spectre- BTI 攻击不同， VMS CAPE无需对任何虚拟机监控程序代码进行修改。VM S CAPE通过一种新颖的vBTI原语实现这一目标：该原语是我们通过对不同AMD和Intel CPU上、不同权限级别下客户机与主机之间分支预测器状态的隔离间隙进行系统性探索而发现的。缓解 VMS CAPE 攻击需要在进入用户空间虚拟机监控程序时执行 VMEXIT 后的 IBPB 操作，这会带来轻微的性能开销。

致谢

我们谨向匿名审稿人及我们的指导老师致以诚挚谢意，感谢他们提供的宝贵反馈。同时，我们也衷心感谢英特尔 PSIRT 、AMD PSIRT 和Linux安全团队在该项目中的协调与合作。此外，特别感谢谷歌的Alexandra Sandulescu对我们缓解方案分析提出的宝贵意见。本研究由瑞士教育、研究与创新部根据合同编号MB22.00057（ERC-StG承诺）资助完成。

References

[1] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom, "Spectre Attacks: Exploiting Speculative Execution," in *2019 IEEE Symposium on Security and Privacy (SP)*, May 2019, pp. 1–19.

[2] J. Wikner and K. Razavi, "RETBLEED: Arbitrary Speculative Code Execution with Return Instructions," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3825–3842.

[3] E. Barberis, P. Frigo, M. Muench, H. Bos, and C. Giuffrida, "Branch History Injection: On the Effectiveness of Hardware Mitigations Against Cross-Privilege Spectre-v2 Attacks," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 971–988.

[4] S. Wiebing, A. d. F. Tron, H. Bos, and C. Giuffrida, "InSpectre Gadget: Inspecting the Residual Attack Surface of Cross-privilege Spectre v2," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 577–594.

[5] S. Wiebing and C. Giuffrida, "Training Solo: On the Limitations of Domain Isolation Against Spectre-v2 Attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, May 2025, pp. 3599–3616.

[6] D. Trujillo, J. Wikner, and K. Razavi, "Inception: Exposing New Attack Surfaces with Training in Transient Execution," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 7303–7320.

[7] J. Wikner and K. Razavi, "Breaking the Barrier: Post-Barrier Spectre Attacks," in *S&P*, May 2025.

[8] S. Rüegge, J. Wikner, and K. Razavi, "Branch Privilege Injection: Compromising Spectre v2 Hardware Mitigations by Exploiting Branch Predictor Race Conditions," in *USENIX Security*, Aug. 2025, intel Bounty Reward, BlackHat USA presentation. [Online]. Available: Paper=https://comsec.ethz.ch/wp-content/files/bprc_sec25.pdfURL=https://comsec.ethz.ch/bprc

[9] M.-M. Bazm, M. Lacoste, M. Südholt, and J.-M. Menaud, "Isolation in cloud computing infrastructures: New security challenges," *Annals of Telecommunications*, vol. 74, no. 3, pp. 197–209, Apr. 2019.

[10] Advanced Micro Devices, Inc., "AMD64 APM, Volume 2: System Programming," Tech. Rep. 24593, Mar. 2024.

[11] "Intel Trust Domain Extension," Tech. Rep.

[12] "Kernel virtual machine," https://linux-kvm.org/page/Main_Page.

[13] "QEMU: A generic and open source machine emulator and virtualizer," https://www.qemu.org//.

[14] "Intel64 SDM, Volume 3 (3A, 3B, 3C & 3D): System Programming Guide," Tech. Rep.

[15] "Proxmox: Simplify your data center," https://proxmox.com/en/.

[16] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. Tullsen, "Pathfinder: High-Resolution Control-Flow Attacks Exploiting the Conditional Branch Predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. La Jolla CA USA: ACM, Apr. 2024, pp. 770–784.

[17] L. Li, H. Yavarzadeh, and D. Tullsen, "Indirector: High-Precision Branch Target Injection Attacks Exploiting the Indirect Branch Predictor," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 2137–2154.

[18] Advanced Micro Devices, Inc., "Software Optimization Guide for the AMD Zen5 Microarchitecture," Tech. Rep. 58455, Aug. 2024.

[19] Y. Zhang and R. Sion, "Speculative Execution Attacks and Cloud Security," in *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*, ser. CCSW'19. New York, NY, USA: Association for Computing Machinery, Nov. 2019, p. 201.

[20] Y. Yarom and K. Falkner, "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack," in *23rd USENIX Security Symposium (USENIX Security 14)*, 2014, pp. 719–732.

[21] E. M. Koruyeh, K. N. Khasawneh, C. Song, and N. Abu-Ghazaleh, "Spectre Returns! Speculation Attacks using the Return Stack Buffer," in *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[22] "Indirect Branch Restricted Speculation," Tech. Rep.

[23] Advanced Micro Devices, Inc., "Amd64 Technology Indirect Branch Control Extension," Tech. Rep., Jul. 2018.

[24] "Single Thread Indirect Branch Predictors," Tech. Rep.

[25] "Providing secret data to QEMU," https://www.qemu.org/docs/master/system/secrets.html.

[26] H. Wang, M. Tang, Q. Wang, K. Xu, and Y. Zhang, "ZenLeak: Practical Last-Level Cache Side-Channel Attacks on AMD Zen Processors," 2025.

[27] Advanced Micro Devices, Inc., "AMD64 APM, Volume 3: General-Purpose and System Instructions," Tech. Rep. 24594, Mar. 2024.

[28] ——, "Software Optimization Guide for the AMD Zen4 Microarchitecture," Tech. Rep. 57647, Jan. 2023.

[29] "Speculative Return Stack Overflow (SRSO) — The Linux Kernel documentation," https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/srso.html.

[30] Advanced Micro Devices, Inc., "Technical Guidance for Mitigating Branch Type Confusion," Tech. Rep.

[31] D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, "Jump over ASLR: Attacking branch predictors to bypass ASLR," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2016, pp. 1–13.

[32] Documentation for /proc/sys/kernel. The Linux Kernel documentation. [Online]. Available: https://docs.kernel.org/admin-guide/sysctl/kernel.html#numa-balancing

[33] Configure NUMA-aware VMs — Google Distributed Cloud (software only) for bare metal. Google Cloud. [Online]. Available: https://cloud.google.com/kubernetes-engine/distributed-cloud/bare-metal/docs/vm-runtime/numa

[34] How ESXi NUMA Scheduling Works. [Online]. Available: https://techdocs.broadcom.com/us/en/vmware-cis/vsphere/vsphere/8-0/vsphere-resource-management-8-0/using-numa-systems-with-esxi/how-esxi-numa-scheduling-works.html

[35] NUMA - Proxmox VE. [Online]. Available: https://pve.proxmox.com/wiki/NUMA

[36] "Branch History Injection and Intra-mode Branch Target Injection," https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/branch-history-injection.html.

[37] "UnixBench test suite," https://github.com/kdlucas/byte-unixbench.

[38] "Flexible I/O Tester," https://github.com/kdlucas/byte-unixbench.

[39] "Retpoline: A software construct for preventing branch-target-injection," https://support.google.com/faqs/answer/7625886.

[40] "VirtualBox: Powerful open source virtualization For personal and enterprise use," https://www.virtualbox.org/.

[41] "Firecracker: Secure and fast microVMs for serverless computing."

[42] "Crosvm: The ChromeOS Virtual Machine Monitor."

[43] J. Wikner, D. Trujillo, and K. Razavi, "Phantom: Exploiting Decoder-detectable Mispredictions," in *56th Annual IEEE/ACM International Symposium on Microarchitecture*. Toronto ON Canada: ACM, Oct. 2023, pp. 49–61.

[44] P. Michaud and A. Seznec, "A Comprehensive Study of Dynamic Global History Branch Prediction," Report, INRIA, 2001.

[45] W.-M. Hu, "Lattice scheduling and covert channels," in *Proceedings 1992 IEEE Computer Society Symposium on Research in Security and Privacy*, 1992, pp. 52–61.

[46] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *Advances in Cryptology — CRYPTO '96*, N. Koblitz, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996, pp. 104–113.

[47] O. Acıiçmez, Ç. K. Koç, and J.-P. Seifert, "Predicting secret keys via branch prediction," in *Topics in Cryptology – CT-RSA 2007*, M. Abe, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 225–242.

[48] O. Aciiçmez, c. K. Koç, and J.-P. Seifert, "On the power of simple branch prediction analysis," in *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*, ser. ASIACCS '07. New York, NY, USA: Association for Computing Machinery, 2007, p. 312–320. [Online]. Available: https://doi.org/10.1145/1229285.1266999

[49] Q. Ge, Y. Yarom, D. Cock, and G. Heiser, "A survey of microarchitectural timing attacks and countermeasures on contemporary hardware," 2018.

[50] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading Kernel Memory from User Space," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 973–990.

[51] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, "Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, p. 991.

[52] V. Kiriansky and C. Waldspurger, "Speculative buffer overflows: Attacks and defenses," 2018. [Online]. Available: https://arxiv.org/abs/1807.03757

[53] J. Stecklina and T. Prescher, "Lazyfp: Leaking fpu register state using microarchitectural side-channels," 2018. [Online]. Available: https://arxiv.org/abs/1806.07480

[54] D. Moghimi, "Downfall: Exploiting speculative data gathering," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 7179–7193. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/moghimi

[55] S. van Schaik, A. Milburn, S. Österlund, P. Frigo, G. Maisuradze, K. Razavi, H. Bos, and C. Giuffrida, "RIDL: Rogue in-flight data load," in *S&P*, May 2019.

[56] C. Canella, D. Genkin, L. Giner, D. Gruss, M. Lipp, M. Minkin, D. Moghimi *et al.*, "Fallout: Leaking data on meltdown-resistant cpus," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2019.

[57] M. Schwarz, M. Lipp, D. Moghimi, J. Van Bulck, J. Stecklina, T. Prescher, and D. Gruss, "ZombieLoad: Cross-privilege-boundary data sampling," in *CCS*, 2019.

[58] C. Canella, J. V. Bulck, M. Schwarz, M. Lipp, B. von Berg, P. Ortner, F. Piessens, D. Evtyushkin, and D. Gruss, "A systematic evaluation of transient execution attacks and defenses," in *28th USENIX Security Symposium (USENIX Security 19)*. Santa Clara, CA: USENIX Association, Aug. 2019, pp. 249–266. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/canella

[59] "Google CPU security research," https://github.com/google/security-research/tree/master/pocs/cpus.

[60] H. Ragab, E. Barberis, H. Bos, and C. Giuffrida, "Rage against the machine clear: A systematic analysis of machine clears and their implications for transient execution attacks," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 1451–1468. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/ragab

[61] G. Maisuradze and C. Rossow, "ret2spec: Speculative execution using return stack buffers," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 2109–2122. [Online]. Available: https://doi.org/10.1145/3243734.3243761

[62] J. Wikner, C. Giuffrida, H. Bos, and K. Razavi, "Spring: Spectre returning in the browser with speculative load queuing and deep stacks," in *WOOT*, 2022.

[63] J. Kim, D. Genkin, and Y. Yarom, "Slap: Data speculation attacks via load address prediction on apple silicon," in *S&P*, 2025.

[64] J. Kim, J. Chuang, D. Genkin, and Y. Yarom, "Flop: Breaking the apple m3 cpu via false load output predictions," in *USENIX Security*, 2025.

天津大学

本科生毕业设计（论文）过程指导记录表

学院	智能与计算学部	专业	软件工程	姓名	杨宇鑫	学号	3022207128
毕设（论文）题目	处理器微架构侧信道漏洞安全测试技术研究					指导教师	鲁赵骏

序号	记录日期	本周工作与下周计划	指导内容
1	2025-12-24	开启毕设工作	读论文文献
2	2026-01-04	本周：读论文文献 下周：学习毕业设计相关的基本原理	论文文献与学习方法
3	2026-01-10	本周：学习侧信道攻击基础理论，了解缓存架构与工作机制 下周：深入学习flush and reload攻击原理	侧信道攻击分类与缓存基本原理
4	2026-01-14	本周：学习了flush and reload 的攻击原理 下周：完成flush and reload攻击代码	分析代码库，使用需要的工具库，攻击的可行性
5	2026-01-21	本周：搭建实验环境，配置Linux内核与必要工具链 下周：编写flush and reload基础代码框架	实验环境配置与代码框架设计
6	2026-01-28	本周：完成了flush and reload的代码 下周：完成具体的攻击	分析了攻击的具体实现
7	2026-02-10	本周：验证flush and reload攻击在本地环境的有效性 下周：优化攻击代码，提高攻击成功率与稳定性	攻击验证方法与性能优化思路
8	2026-02-24	本周：研究Prime+Probe等其他缓存侧信道攻击方法 下周：对比不同攻击方法，确定最终方案	多种侧信道攻击方法对比与方案选型
9	2026-03-05	本周：分析DES加密算法在侧信道攻击下的脆弱性 下周：实现flush and reload对DES算法的攻击	DES算法结构分析与攻击点识别
10	2026-03-12	本周：完成了flush and reload对DES算法加密的攻击 下周：修改为对AES算法加密的攻击	DES算法在很多领域已经过时，攻击AES的价值更高
11	2026-03-25	本周：完成了flush and reload对AES128 AES192 AES256加密的攻击 下周：完成中期报告	完成中期报告与毕设论文的目录
12	2026-04-08	本周：整理AES攻击实验数据，分析不同密钥长度下的攻击成功率 下周：撰写论文引言与相关工作章节	实验数据分析方法与论文框架设计
13	2026-04-22	本周：撰写论文核心章节（攻击原理与实现细节）< 下周：撰写实验设计与结果分析章节	论文技术内容写作与图表规范
14	2026-05-06	本周：完成论文实验结果分析与对比讨论，完善结论 下周：修改论文格式，准备查重	论文整体逻辑梳理与学术规范
15	2026-05-20	本周：完成论文终稿，制作答辩PPT 下周：进行答辩演练与最终修改	答辩技巧与常见问题准备

指导教师签名：鲁赵骏

天津大学

本科生毕业设计（论文）中期检查记录表

学院	智能与计算学部	专业	软件工程
学生姓名	杨宇鑫	学号	3022207128
指导教师	鲁赵骏	选题是否有变化	□是 ☑否
课题名称	处理器微架构侧信道漏洞安全测试技术研究		
毕业设计（论文）进度情况		□提前 ☑正常 □滞后	
对是否可以按期完成毕业设计（论文）的评估		☑是 □否	
已完成的研究工作及成果	已成功实现了针对 AES-128、AES-192 和 AES-256 三种密钥长度的真实 Flush+Reload 缓存侧信道攻击系统。 在攻击原理研究与实现方面，深入分析了 AES 算法中 T 表查找操作引入的缓存侧信道泄露机制。攻击利用 AES 加密过程中对 T 表的访问模式与明文、密钥之间的相关性，通过监控特定缓存行的访问状态来推断最后一轮密钥。系统采用 POSIX 共享内存机制实现跨进程 T 表共享，使用 fork()创建 victim 进程和 attacker 进程，利用 x86 架构的 clflush 指令精确控制缓存状态，并通过 RDTSC 指令实现纳秒级的时间测量，从而准确区分缓存命中和未命中事件。 在攻击算法实现方面，开发了基于排除法的密钥恢复算法。对于每个加密样本，如果检测到 T 表缓存行未被访问，则可以排除与该缓存行相关的 16 个密钥候选值。通过大量样本的累积统计，选择被排除次数最少的候选值作为正确的密钥字节。针对 AES-128，实现了从最后一轮密钥 K10 逆向推导原始密钥 K0 的完整流程；针对 AES-192 和 AES-256，设计了基于 EBD 方法的扩展攻击方案。 在实验验证方面，建立了完整的攻击测试框架，支持自动化运行攻击实验并收集统计结果。实验数据显示，AES-128 在 8000 样本条件下攻击成功率达到 99%，AES-192 在 10000 样本条件下成功率达到 96%，AES-256 在 30000 样本条件下成功率达到 99%，充分验证了攻		

	击方法的有效性。
下一步工作计划	下一阶段的研究工作将主要围绕缓解措施对比实验和攻击指标可视化两个方面展开。 在缓解措施对比实验方面，计划设计并实现多种针对 Flush+Reload 攻击的防御机制，包括噪声注入（低/中/高三级）、缓存刷新、CPU 核心绑定控制等策略。通过系统化对比实验，评估各缓解措施在不同 AES 类型和样本数量条件下的攻击抑制效果，测量攻击成功率和性能开销，建立安全性与性能的权衡关系。 在攻击指标可视化方面，将开发多维度的数据可视化工具，包括缓存访问时间分布图、信噪比变化曲线、互信息热力图等，直观展示攻击过程中的缓存侧信道泄露特征及缓解措施的抑制效果。
存在问题与解决方案	存在问题：攻击成功率存在波动： 受系统负载等因素影响，成功率不够稳定。 解决方案：优化时间测量精度，实现动态阈值校准机制，增加多轮投票机制提高可靠性。

指导教师意见	尽快完成实验部分的任务，开始撰写毕业论文。 签字：  2026年 4月 1日
系（教研室） 审核意见	该生研究进展突出，已成功实现AES-128/192/256全系列Flush+Reload攻击系统，攻击成功率均达95%以上，实验数据翔实，充分验证了方案有效性。对缓存侧信道泄露机理分析深入，算法实现规范。下一步拟开展的缓解措施对比与可视化研究思路清晰，有望取得创新性成果。工作进度符合预期，表现出优秀的科研能力与严谨态度。 签字：  2026年 4月 1日

天津大学

本科生毕业设计（论文）指导教师评阅书

智能与计算学部 学院　软件工程 专业　2022 年级　学生 杨宇鑫

毕业设计（论文）题目	处理器微架构侧信道漏洞安全测试技术研究		
课题来源	其他		
课题类型	A.设计类☑ B.论文类☐ C.软件类☐		
指导教师姓名	鲁赵骏	职　称	副教授
开题报告成绩（百分制）	95	设计（论文）成绩（百分制）	94

开题报告立论充分，面向处理器微架构安全这一芯片安全核心领域，选题具有重要的学术价值。研究内容系统深入，设计了涵盖缓存侧信道攻击（Flush+Reload）的自动化安全测试框架，实现了从漏洞检测到量化评估的完整流程。外文文献覆盖前沿研究，翻译准确。工作量充实，涉及 AES 多种变体的攻击复现与缓解措施验证。工作态度严谨，论文质量较高，创新性体现在统一化、自动化的安全测试框架设计及量化评估指标体系。应用性强，可直接用于芯片安全测试。论文写作规范，逻辑严密。不足之处在于第二章篇幅偏长，缺少必要原理图辅助说明；实验部分缺乏与国内外同类工作的横向对比分析。综合评价：论文整体水平优秀，同意提交答辩。

指导教师（签字）：鲁赵骏

2026年　5月　20日

天津大学

本科生毕业设计（论文）评阅教师评阅书

智能与计算学部 学院 软件工程 专业 2022 年级 学生 杨宇鑫

毕业设计（论文）题目	处理器微架构侧信道漏洞安全测试技术研究		
评阅教师姓名	马哲	职 称	讲师
设计（论文）成绩（百分制）	80		

本文以处理器缓存侧信道攻击技术为主题，选题明确，具有重要的研究价值。总体研究思路明确，工作量充足，内容准确完整。建议针对以下问题进行修改：

1. 开题报告国内外研究进展总结过于简洁； 2. 全文口语化表述明显，建议消除“科普文”语气； 3. 摘要不需过度分段； 4. 公式缺少编号，请补充； 5. 正文国内外研究现状缺少本文创新性的分析； 6. 第二页排版出现超出边界的情况，建议调整； 7. 未提交外文翻译。

评阅教师（签字）：马哲

2026 年 6 月 1 日

天津大学

本科生毕业设计（论文）答辩记录书

智能与计算学部学院　软件工程专业　2022年级　学生杨宇鑫

毕业设计(论文)题目	处理器微架构侧信道漏洞安全测试技术研究				
答辩委员会主席	何家骥	职称	副教授	答辩委员会秘书	杨葛英
答辩委员会成员	张蕾，马哲，贾云刚				

开题报告评分（占5%）	指导教师评分（占25%）	评阅教师评分（占20%）	答辩委员会评分（占50%）	设计（论文）总成绩
95	94	80	85	86.75

1. 综合评价

该生选题紧密结合信息安全领域实际需求，围绕处理器微架构侧信道漏洞安全测试展开研究，具有理论价值和应用前景。论文系统梳理了Flush+Reload攻击机制，设计了支持AES-128/192/256的统一攻击框架与两阶段密钥恢复策略，建立了量化评估指标体系。实验对噪声注入与缓存刷新两种缓解措施进行了系统对比，数据详实，结论可信。论文结构完整，逻辑清晰，答辩过程中能够准确回答评委提出的问题。同意通过答辩。

答辩委员会主席（签字）：何家骥

2026年　6月　3日

2. 答辩记录

(1) 陈述环节

研究背景：云计算多租户环境下微架构侧信道威胁日益突出。研究问题：如何对AES加密实现Flush+Reload缓存侧信道攻击进行系统化安全测试与量化评估。方法论：基于排除法设计统一攻击框架，支持AES-128/192/256三阶段密钥恢复；建立命中率、SNR、泄露带宽等量化指标。实验结果：基准配置下三种AES变体攻击成功率均为100%；缓存刷新可完全阻断攻击，噪声注入效果随密钥长度增加而提升。结论：为云平台侧信道安全测试提供了可落地的方法论与工具。

(2) 问答环节

Q1：为什么选择AES加密算法作为攻击目标？

A：AES是目前应用最广泛的对称加密标准，其实现中的T表查表操作会产生规律的缓存访问痕迹，非常适合验证Flush+Reload攻击与测试方法。

Q2：论文中的"排除法"核心思想是什么？

A：通过观测 T 表缓存行是否被访问来排除错误密钥候选。若某缓存行未被命中，则所有会导致访问该行的密钥值都被排除；多样本累积后，排除计数最少的即为正确密钥字节。

Q3：实验结果表明哪种缓解措施最有效？

A：缓存刷新最有效，能将攻击成功率与泄露带宽全部压到零；噪声注入虽能干扰，但 AES-128 在高噪声下仍有 3%成功率，防护不如缓存刷新彻底。

答辩委员会秘书（签字）：杨葛英

2026 年 6 月 3 日

院长（签章）

2026 年 6 月 3 日